

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук _____
(повна назва)

Кафедра _____ програмної інженерії _____
(повна назва)

КУРСОВА РОБОТА (ПРОЄКТ)

Пояснювальна записка

з дисципліни _____ комплексний курсовий проєкт _____
(назва навчальної дисципліни)

Веб-застосунок для оцінювання та рецензування мультимедійного контенту
(тема роботи (проєкту))

Виконав:

здобувач 3 року навчання, групи ПЗПІ-23-4

Олександр Єрмоленко

(власне ім'я, прізвище)

Керівник доц. Анастасія Чуприна

(посада, власне ім'я, прізвище)

Національна шкала _____

Кількість балів: _____

Оцінка ЄКТС _____

Члени комісії: доц. Лещинський В.О.

(посада, власне ім'я, прізвище)

доц. Мельникова Р.В.

(посада, власне ім'я, прізвище)

ст. викл. Новіков Ю.С.

(посада, власне ім'я, прізвище)

Харків 2026 р.

Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук
Кафедра _____ програмної інженерії
Рівень вищої освіти _____ перший (бакалаврський)
Спеціальність _____ F2 – Інженерія програмного забезпечення
Тип програми _____ освітньо-наукова програма
Освітня програма _____ Інженерія програмного забезпечення
(шифр і назва)

ЗАВДАННЯ

на курсовий проєкт (роботу) здобувачеві

Єрмоленку Олександр Станіславовичу

(прізвище, ім'я, по батькові)

1. Тема роботи _____ «Веб-застосунок для оцінювання та рецензування мультимедійного контенту» _____

2. Термін здачі закінченої роботи 10.06.2026 р.

3. Вихідні дані до роботи: опис розробки клієнтської частини веб-застосунку для оцінювання та рецензування мультимедійного контенту; вимоги до проєктування користувацького інтерфейсу та взаємодії з серверною частиною (API); мови програмування TypeScript; бібліотека React для створення інтерфейсу користувача; технології HTML5, CSS3; середовище розробки Visual Studio Code.

4. Зміст пояснювальної записки

Аркуш завдання, Use-case діаграма, UML-діаграма компонентів, UML-діаграма розгортання, SRS, клієнтська частина програмної системи, графічні матеріали.

5. Перелік графічного матеріалу:

Use-case діаграма, UML-діаграма компонентів, UML-діаграма розгортання.

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів роботи	Термін виконання етапів роботи	Примітка
1	Аналіз предметної галузі	16.02.2026	виконано
2	Розробка постановки задачі	16.03.2026	виконано
3	Проектування ПЗ	23.03.2026	виконано
4	Програмна реалізація	23.03.2026	виконано
5	Аналіз результатів	16.03.2026	виконано
6	Підготовка пояснювальної записки	25.05.2026	виконано
7	Перевірка на наявність ознак академічного плагіату	10.06.2026	виконано
8	Захист роботи	10.06.2026	виконано

Дата видачі завдання 16.02.2026р.

Здобувач

_____ (підпис)

Керівник роботи

_____ (підпис)

доц. Анастасія Чуприна.

(посада власне ім'я прізвище)

РЕФЕРАТ / ABSTRACT

Пояснювальна записка містить: 62 стор., 25 рис., 1 табл., 6 джерел.

ВЕБ-ЗАСТОСУНОК, МУЛЬТИМЕДІЙНИЙ КОНТЕНТ, ОЦІНЮВАННЯ, РЕЦЕНЗУВАННЯ, JAVASCRIPT, TYPESCRIPT, REACT, VITE, HTML5, CSS3, VISUAL STUDIO CODE

Об'єкт розробки - клієнтська частина веб-застосунку для оцінювання та рецензування мультимедійного контенту.

Мета розробки - створення клієнтської частини веб-застосунку RateIt, який надає користувачам можливість оцінювати та рецензувати мультимедійний контент, зокрема фільми, серіали, музику, відеоігри та книги.

Метод рішення - середовище розробки Visual Studio Code, бібліотека React із використанням збірника Vite, мови програмування TypeScript, технології HTML5 та CSS3.

У результаті розробки релізовано клієнтську частину веб-застосунку RateIt, що забезпечує зручний та інтуїтивно зрозумілий користувацький інтерфейс для перегляду, пошуку, оцінювання та рецензування мультимедійного контенту з підтримкою взаємодії із серверною частиною через API.

WEB APPLICATION, MULTIMEDIA CONTENT, RATING, REVIEWING, JAVASCRIPT, TYPESCRIPT, REACT, VITE, HTML5, CSS3, VISUAL STUDIO CODE

The development project is the client-side component of a web application for rating and reviewing multimedia content.

The goal of the project is to create the client-side component of the RateIt web application, which allows users to rate and review multimedia content, including movies, TV shows, music, video games, and books.

Solution approach: Visual Studio Code development environment, React library using the Vite build tool, TypeScript programming language, HTML5, and CSS3.

As a result of the development, the client-side of the RateIt web application was released, providing a convenient and intuitive user interface for viewing, searching, rating, and reviewing multimedia content with support for interaction with the server-side via an API.

ЗМІСТ

Перелік скорочень	8
Вступ	9
1 Аналіз предметної галузі	10
1.1 Аналіз предметної галузі	10
1.2 Виявлення та вирішення проблем	10
1.2.1 Цільова аудиторія	11
1.3 Аналіз аналогів програмного забезпечення.....	12
1.3.1 IMDb	12
1.3.2 Letterboxd	13
1.3.3 Metacritic	13
1.3.4 Висновки аналізу аналогів програмного забезпечення	14
2 Постановка задачі	15
3 Архітектура та проєктування програмного забезпечення	17
3.1 UML проєктування ПЗ	17
3.2 Проєктування архітектури ПЗ	18
4 Опис прийнятих програмних рішень	22
4.1 Реалізація автентифікації та реєстрації користувача	22
4.2 Реалізація перегляду та пошуку контенту	24
4.3 Залишення оцінки та відгуків про контент	28
4.4 Залишення оцінки та відгуків про контент	28
4.4.1 Додавання контенту	32
4.4.2 Редагування контенту	34
4.4.3 Видалення контенту	35
5 Аналіз отриманих результатів.....	37
6 Тестування програмного забезпечення	38
6.1 Тестування веб-застосунку RateIt	38
6.2 Приклади критичних та незначних помилок	39
Висновки	41
Перелік джерел посилання	43

Додаток А	44
Додаток Б	47
Додаток В	53

ПЕРЕЛІК СКОРОЧЕНЬ

API – Application Programming Interface

CSS – Cascading Style Sheets

HTML – HyperText Markup Language

HTTPS – HyperText Transfer Protocol Secure

HTTP – HyperText Transfer Protocol

JSON – JavaScript Object Notation

JWT – JSON Web Token

REST – Representational State Transfer

SPA – Single Page Application

UML – Unified Modeling Language

URL – Uniform Resource Locator

UI – User Interface

UX – User Experience

ВСТУП

Розвиток інформаційних технологій та поширення цифрових сервісів суттєво вплинули на використання мультимедійних ресурсів. Одним із найбільш поширених видів цифрового дозвілля сьогодні є споживання мультимедійного контенту, до якого належать фільми, серіали, музика, відеоігри та книги.

Зі збільшенням кількості доступного контенту зростає потреба в ефективних інструментах, які дозволяють користувачам оцінювати його якість, обмінюватися враженнями. Одним із найпоширеніших інструментів для реалізації такої взаємодії є веб-застосунки, які дозволяють користувачам переглядати інформацію про різні категорії контенту, залишати власні оцінки й рецензії, а також отримувати відгуки від інших користувачів.

Темою комплексного курсового проєкту є розробка клієнтської частини веб-застосунку для оцінювання та рецензування мультимедійного контенту. Основним завданням застосунку є надання користувачеві зручного інтерфейсу для перегляду контенту, виставлення оцінок та написання рецензій.

Метою роботи є розробка клієнтської частини веб-застосунку RateIt, що реалізує повноцінний користувацький інтерфейс із можливістю перегляду контенту, виставлення оцінок та написання рецензій. Для досягнення поставленої мети використовується бібліотека React, збірник Vite, мова програмування TypeScript, а також технології HTML5 та CSS3. Розробка здійснюється у середовищі Visual Studio Code. Взаємодія з даними реалізується через API.

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ

1.1 Аналіз предметної галузі

Розвиток інтернету суттєво змінив способи, в які люди взаємодіють із контентом. Користувачі щоденно переглядають фільми та серіали, слухають музику, грають у відеоігри та читають книги, які є доступними на чисельних платформах. У зв'язку з цим у пересічних користувачів виникає потреба у веб-застосунку, який дозволяв би орієнтуватися в такому різноманітті контенту, отримувати рекомендації та ділитися власними враженнями з іншими користувачами.

Платформи для оцінювання медіа фактично стали невід'ємною частиною сучасної культури споживання контенту. Вони дають змогу не лише виставляти оцінки фільмам чи книгам, а й ознайомитися з відгуками інших користувачів або знаходити щось нове на основі рекомендацій спільноти.

Більшість існуючих платформ орієнтовані лише на один тип контенту, тому користувачам доводиться використовувати декілька різних сервісів одночасно. Саме тому актуальним є створення універсальної платформи, яка об'єднує оцінювання різних типів мультимедійного контенту в єдиному інтерфейсі.

Практична цінність проєкту RateIt полягає у створенні веб-застосунку, що поєднує функціонал платформи для оцінювання та рецензування фільмів, серіалів, музики, відеоігор і книг. Завдяки зручному інтерфейсу та взаємодії із серверним API користувач отримує доступ до актуальної інформації про контент, може залишати рецензії та переглядати оцінки інших учасників спільноти.

1.2 Виявлення та вирішення проблем

Основною проблемою є розпорошеність інформації між різними платформами: для оцінювання фільмів використовується один сервіс, для музики - інший, для ігор - третій застосунок. Це створює незручності та змушує витратити час на перехід між різними застосунками. Більшість платформ не надають можливості вести єдиний список переглянутого, прочитаного та прослуханого

матеріалу. Це ускладнює відстеження власних уподобань та формування персональних рекомендацій.

Варто зазначити, що у багатьох сучасних платформах користувацькі інтерфейси мають низьку якість. Застарілий дизайн, складна навігація та відсутність адаптивності для мобільних пристроїв негативно впливають на зручність використання сервісу.

Запропонований проєкт RateIt спрямований на вирішення зазначених проблем шляхом розробки клієнтської частини веб-застосунку з єдиним інтерфейсом для всіх категорій контенту. Використання сучасного стеку технологій: React, TypeScript, Vite, HTML5 та CSS3 - дозволяє створити швидкий, адаптивний та зручний інтерфейс, що забезпечує комфортну взаємодію користувача з платформою незалежно від пристрою.

1.2.1 Цільова аудиторія

Цільовою аудиторією проєкту є активні споживачі мультимедійного контенту віком від 16 до 55 років, які регулярно переглядають фільми та серіали, слухають музику, грають у відеоігри та читають книги. Це люди, яким важливо мати зручний інструмент для відстеження переглянутого контенту та обміну враженнями з іншими користувачами.

Окрему частину аудиторії становлять користувачі, які активно шукають рекомендації перед вибором нового фільму, гри чи книги та орієнтуються на думку спільноти. Для них важлива присутність зручного пошуку, системи фільтрації та можливості ознайомлення з рецензіями, які залишили інші користувачі.

Також до цільової аудиторії належать критики та блогери, які публікують рецензії та формують власну аудиторію читачів. Для них платформа є майданчиком для самовираження, критики та обговорення мультимедійного контенту.

1.3 Аналіз аналогів програмного забезпечення

Ринок платформ для оцінювання та рецензування мультимедійного контенту пропонує користувачам широкий вибір сервісів. Основною метою таких платформ є надання зручного інструменту для пошуку інформації про контент, ознайомлення з оцінками та рецензіями користувачів спільноти. Для аналізу конкурентного середовища розглянемо найпопулярніші платформи: IMDb, Letterboxd, Metacritic, а також їхні основні переваги та недоліки.

1.3.1 IMDb

IMDb (Internet Movie Database) є найбільшою у світі базою даних фільмів та серіалів. Платформа надає користувачам доступ до детальної інформації про кінострічки, акторський склад, режисерів та знімальну групу. Користувачі можуть виставляти оцінки за шкалою від 1 до 10, писати рецензії та формувати списки улюблених фільмів. IMDb також надає доступ до рейтингів, які формуються на основі голосів користувачів, зокрема відомий список IMDb Top 250.

До переваг платформи належать величезна база даних, висока впізнаваність серед користувачів та детальна інформація про знімальні групи. Проте IMDb орієнтований виключно на кіно та телебачення і не охоплює інші категорії контенту, такі як музику, ігри чи книги.

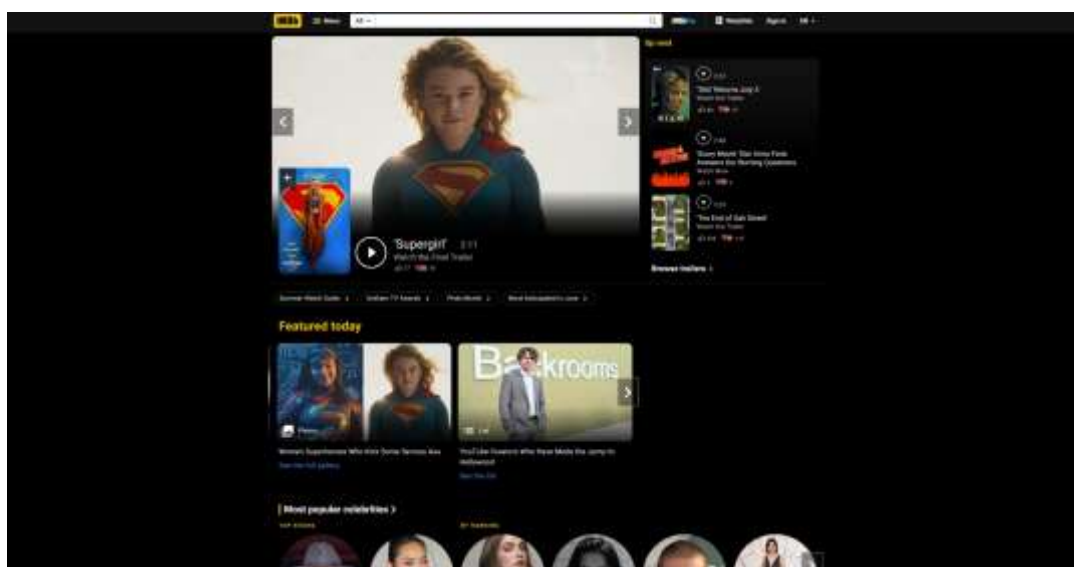


Рисунок 1.1 – Інтерфейс платформи IMDb

1.3.2 Letterboxd

Letterboxd є соціальною мережею для любителів кіно, орієнтованою на ведення особистих щоденників перегляду фільмів. Платформа дозволяє користувачам виставляти оцінки, писати рецензії, формувати списки фільмів та стежити за активністю інших учасників спільноти. Letterboxd відзначається лаконічним сучасним інтерфейсом та сильною спільнотою кіноманів.

Перевагою платформи є висока залученість спільноти та зручний соціальний функціонал. Водночас Letterboxd обмежений виключно кіноконтентом і не підтримує інші категорії медіа, що звужує його аудиторію.

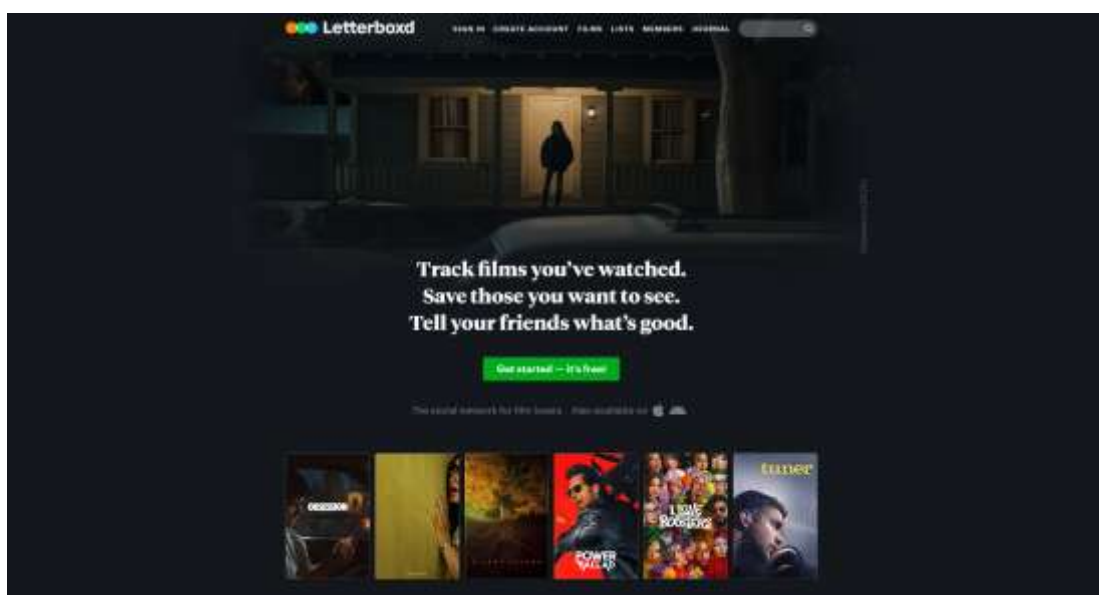


Рисунок 1.2 – Інтерфейс платформи Letterboxd

1.3.3 Metacritic

Metacritic є агрегатором рецензій, який збирає оцінки від професійних критиків та звичайних користувачів для фільмів, серіалів, музичних альбомів та відеоігор. Головною особливістю платформи є система Metascore — зважена середня оцінка на основі рецензій авторитетних видань. Це дозволяє користувачам отримати об'єктивніший погляд на якість контенту порівняно з платформами, що спираються виключно на думку звичайних користувачів.

Перевагою Metacritic є охоплення кількох категорій контенту та наявність професійних рецензій. Недоліком є складний та візуально перевантажений інтерфейс, а також відсутність розділу для книг та обмежені соціальні функції для користувачів.

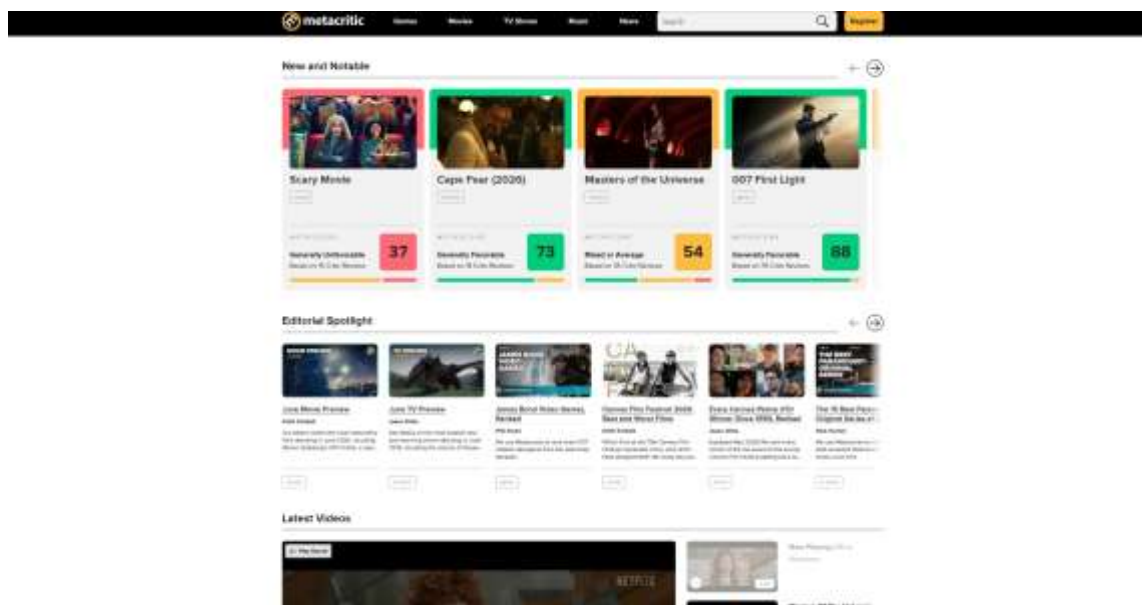


Рисунок 1.3 – Інтерфейс платформи Metacritic

1.3.4 Висновки аналізу аналогів програмного забезпечення

Аналіз існуючих платформ свідчить, що більшість із них орієнтовані на один тип контенту та не забезпечують оцінювання різних категорій медіа. Платформа Metacritic є наближеним аналогом за охопленням категорій, однак поступається за зручністю інтерфейсу та рівнем соціальних взаємодій. Платформа Letterboxd є найкращим прикладом сучасного UX-дизайну, проте обмежується виключно кіно.

Усі зазначені аспекти підтверджують доцільність розробки веб-застосунку RateIt, який поєднує переваги існуючих платформ - широке охоплення категорій контенту, зручний сучасний інтерфейс та соціальний функціонал - в межах єдиної платформи.

2 ПОСТАНОВКА ЗАДАЧІ

Аналіз існуючих платформ сучасного цифрового середовища виявив низку недоліків, зокрема обмежений функціонал, складний інтерфейс, орієнтованість лише на окремі категорії контенту та відсутність єдиного профілю користувача. Саме це визначило основну задачу проєкту - створення універсального веб-застосунку, здатного усунути виявлені недоліки та забезпечити зручний доступ до функцій оцінювання, рецензування й обговорення мультимедійного контенту.

Проєкт RateIt орієнтований на комплексне вирішення зазначених проблем і створення єдиного середовища для взаємодії користувачів із різними видами медіаконтенту. RateIt функціонує як універсальна платформа зі зручним інтерфейсом, що дозволяє переглядати інформацію про фільми, серіали, музику, відеоігри та книги, виставляти оцінки та писати рецензії, фільтрувати контент за категоріями та жанрами.

Ключовими аспектами клієнтської частини системи є:

- адаптивний інтерфейс, який коректно відображається на різних пристроях;
- зручна навігація між категоріями контенту з підтримкою пошуку та фільтрації;
- інтеграція з серверною частиною через API для отримання та взаємодії з даними про контент;
- підтримка облікових записів користувачів для збереження відгуків та оцінок;
- можливість додавання, редагування та видалення контенту з бази даних;
- залучення користувачів до взаємодії з платформою за допомогою системи досягнень;
- розмежування прав доступу між звичайними користувачами та адміністраторами системи для забезпечення керування контентом і модерації платформи.

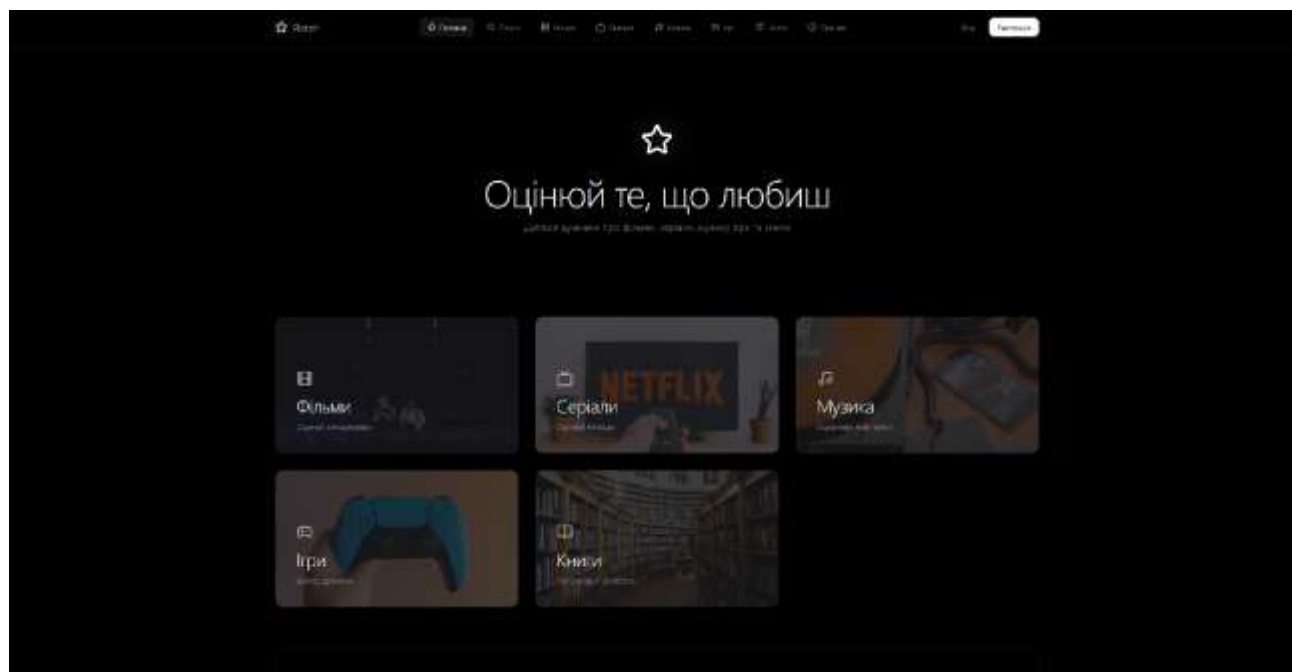


Рисунок 2.1 – Інтерфейс головної сторінки RateIt

Отже, у межах курсової роботи поставлено завдання розробити зручний та інтуїтивно зрозумілий інтерфейс клієнтської частини веб-застосунку RateIt, який забезпечує можливість оцінювання мультимедійного контенту, залишення рецензій та перегляду відгуків інших користувачів. Це дозволить підвищити рівень користувацької активності та створити стійку спільноту, об'єднану інтересом до різноманітного медіаконтенту.

3 АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 UML проєктування ПЗ

Перед початком розробки веб-застосунку були визначені основні функції зі сторони неавторизованого та авторизованого користувача, модератора. Діаграма Use-case (див. рис. 3.1) відображає взаємодію користувачів різних ролей із системою.



Рисунок 3.1 – Use-case діаграма веб-застосунку RateIt

У системі RateIt визначено три категорії користувачів із різним рівнем доступу до функціоналу: неавторизований користувач, авторизований користувач та модератор.

Неавторизований користувач це відвідувач платформи, який ще не пройшов процедуру аутентифікації. Він має можливість:

- переглядати інформацію про мультимедійний контент (фільми, серіали, музику, ігри, книги);
- переглядати оцінки та рецензії інших користувачів;
- здійснювати пошук контенту та застосовувати фільтри за категоріями, жанрами тощо;
- зареєструватися в системі за допомогою електронної пошти.

Авторизований користувач це зареєстрований учасник платформи, який пройшов процедуру аутентифікації. Він має можливість:

- авторизуватися в системі під своїм обліковим записом;
- виставляти оцінки залишати рецензії на мультимедійного контенту;
- керувати власним профілем, зокрема змінювати персональні дані.

Модератор це користувач системи з розширеними правами доступу, відповідальний за забезпечення якості контенту на платформі. Він має можливість:

- авторизуватися в системі під своїм обліковим записом;
- додавати новий мультимедійний контент до бази даних платформи;
- модерувати наявний контент, зокрема редагувати або видаляти некоректні дані.

Use-case діаграма демонструє основні сценарії взаємодії кожного типу користувача із системою RateIt та чітко розмежовує рівні доступу до функціоналу. Побудована діаграма є основою для проєктування архітектури клієнтської частини застосунку.

3.2 Проєктування архітектури ПЗ

Архітектурна модель клієнтської частини побудована за принципом клієнт-серверної архітектури. Фронтенд реалізовано у вигляді односторінкового

застосунку (SPA) з використанням бібліотеки React та мови програмування TypeScript. Збірка проєкту здійснюється за допомогою інструменту Vite, що забезпечує високу швидкість розробки та оптимізовану збірку.

Основними компонентами клієнтської архітектури є: модуль автентифікації, модуль перегляду та пошуку контенту, модуль відгуків та оцінювання, модуль керування профілем, а також панель модератора. Модуль автентифікації відповідає за реєстрацію та вхід користувачів із збереженням JWT-токена у localStorage браузера. Модуль перегляду та пошуку забезпечує відображення контенту з підтримкою фільтрації та сортування. Модуль відгуків реалізує функціонал виставлення оцінок та написання рецензій. Модуль керування профілем надає користувачеві можливість редагувати персональні дані. Панель модератора забезпечує інструменти для додавання, редагування та видалення контенту.

Механізми комунікації між клієнтською та серверною частинами реалізуються через REST API. Для надсилання HTTP-запитів використовується бібліотека axios, яка забезпечує автоматичне додавання JWT-токена до заголовків запитів через інтерцептор. Передача даних здійснюється за протоколом HTTPS у форматі JSON, що гарантує безпечність та захист від несанкціонованого доступу.

Для візуалізації структури клієнтської частини системи побудовано UML-діаграму компонентів, яка відображає основні модулі застосунку та їхні взаємозв'язки (див. рис. 3.2).

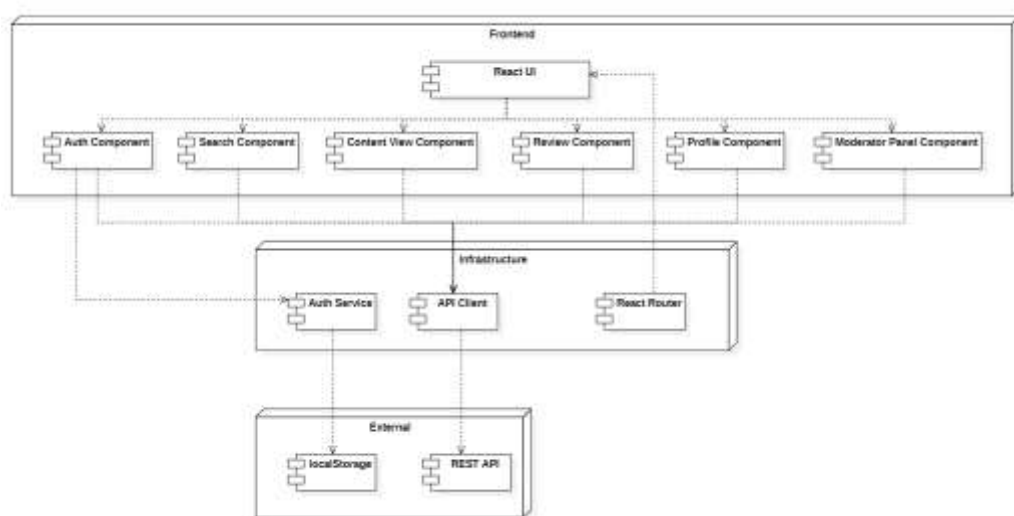


Рисунок 3.2 – UML-діаграма компонентів веб-застосунку RateIt

Окрім цього, побудовано UML-діаграму розгортання, яка показує розміщення компонентів системи на відповідних вузлах: клієнтський браузер із React SPA, фронтенд-сервер на базі Node.js та Vite, серверний API та база даних (див. рис. 3.3).

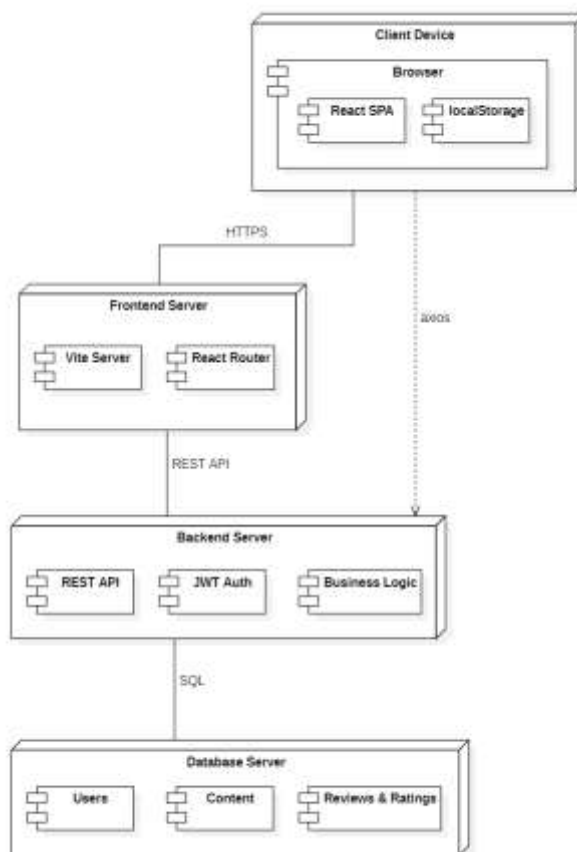


Рисунок 3.3 – UML-діаграма розгортання веб-застосунку RateIt

Нефункціональні вимоги до клієнтської частини системи включають високу швидкість завантаження інтерфейсу, адаптивність для коректного відображення на різних пристроях та браузерах, надійність збереження стану автентифікації користувача, а також захищеність персональних даних. Для виконання цих вимог передбачено використання оптимізованої збірки Vite, механізму збереження JWT-токена у **localStorage**, а також передачу даних виключно через захищений протокол **HTTPS**.

Серед основних ризиків клієнтської частини можна виділити можливі проблеми із сумісністю в різних браузерах, втрату стану автентифікації при

очищенні даних браузера користувачем, а також ризик несанкціонованого доступу до збережених даних. Для зменшення цих ризиків передбачено тестування у різних браузерах, реалізацію механізму автоматичного оновлення токена та використання захищених HTTP-заголовків.

4 ОПИС ПРИЙНЯТИХ ПРОГРАМНИХ РІШЕНЬ

4.1 Реалізація автентифікації та реєстрації користувача

Під час відкриття веб-застосунку RateIt користувач переходить на головну сторінку, на якій розміщено навігаційну панель із кнопкою входу (див. рис. 4.1).

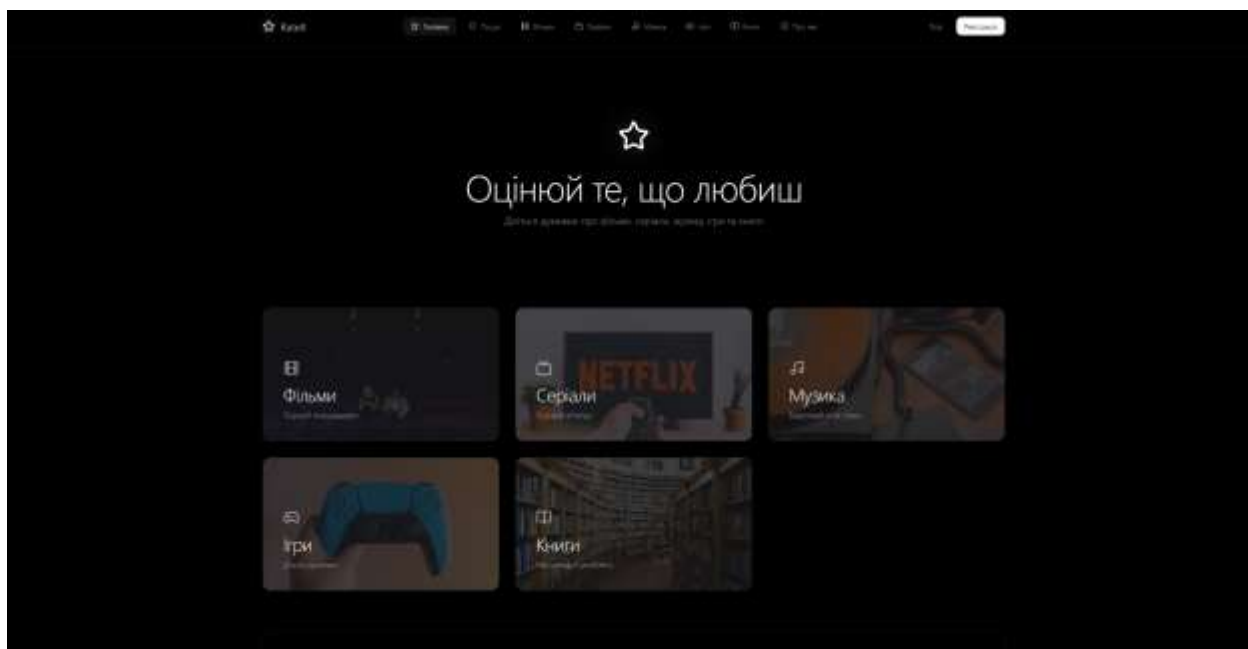
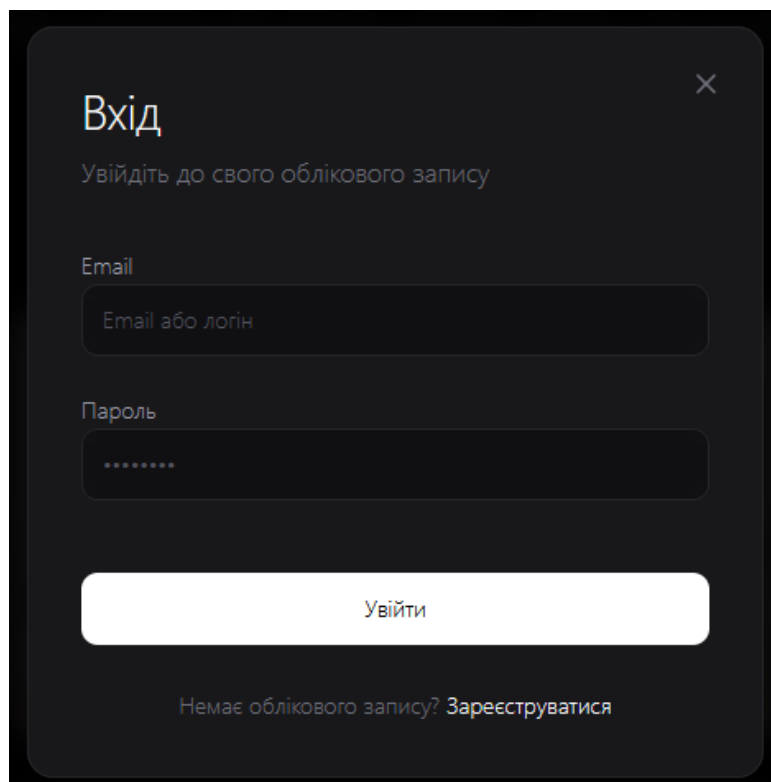


Рисунок 4.1 – Головна сторінка застосунку RateIt

Для входу в систему користувач натискає кнопку «Вхід», після чого відкривається форма автентифікації (див. рис. 4.2). Форма містить два поля для введення даних: електронну пошту та пароль. Після заповнення полів користувач підтверджує введені дані натисканням кнопки «Увійти», і клієнтська частина застосунку формує POST-запит до серверного API з передачею введених даних у форматі JSON. На стороні сервера виконується перевірка облікових даних, і у разі успішної аутентифікації повертається JWT-токен. Отриманий токен зберігається у localStorage браузера та використовується для авторизації подальших запитів до API. Після успішного входу користувач автоматично перенаправляється на головну сторінку застосунку з оновленим станом інтерфейсу.



The image shows a login form with a dark background. At the top left is the title 'Вхід' (Login) in white, and at the top right is a close button 'X'. Below the title is the instruction 'Увійдіть до свого облікового запису' (Log in to your account) in a lighter gray. There are two input fields: 'Email' with the placeholder 'Email або логін' (Email or login) and 'Пароль' (Password) with a masked password '.....'. Below these fields is a large white button labeled 'Увійти' (Log in). At the bottom, there is a link 'Немає облікового запису? Зареєструватися' (Don't have an account? Register) in a lighter gray.

Рисунок 4.2 – Форма входу в систему

Для входу в систему користувач натискає кнопку «Реєстрація», після чого відкривається форма реєстрації (див. рис. 4.3). Форма містить такі поля для заповнення:

- ім'я користувача;
- прізвище користувача;
- вік;
- електронна пошта;
- пароль.

Після заповнення всіх обов'язкових полів користувач натискає кнопку «Зареєструватися», після чого клієнтська частина надсилає POST-запит на сервер із введеними даними у форматі JSON. У разі успішної реєстрації обліковий запис створюється в базі даних, а користувач перенаправляється на сторінку входу для подальшої автентифікації.

The image shows a dark-themed registration form titled "Реєстрація" (Registration) with a close button (X) in the top right corner. Below the title is the subtitle "Створіть новий обліковий запис" (Create a new account). The form contains five input fields: "Ім'я" (Name) with placeholder "Ваше ім'я", "Прізвище" (Surname) with placeholder "Ваше прізвище", "Вік" (Age) with placeholder "Ваш вік" and a dropdown arrow, "Email" with placeholder "your@email.com", and "Пароль" (Password) with placeholder "*****". At the bottom is a large white button labeled "Зареєструватися" (Register). Below the button is a link: "Вже є обліковий запис? Увійти" (Already have an account? Log in).

Рисунок 4.3 – Форма реєстрації нового користувача

4.2 Реалізація перегляду та пошуку контенту

Для пошуку необхідного контенту користувач переходить на сторінку пошуку (див. рис. 4.4), де розміщено поле введення пошукового запиту. Після введення назви контенту клієнтська частина надсилає запит до серверного API та відображає відповідні результати на сторінці.

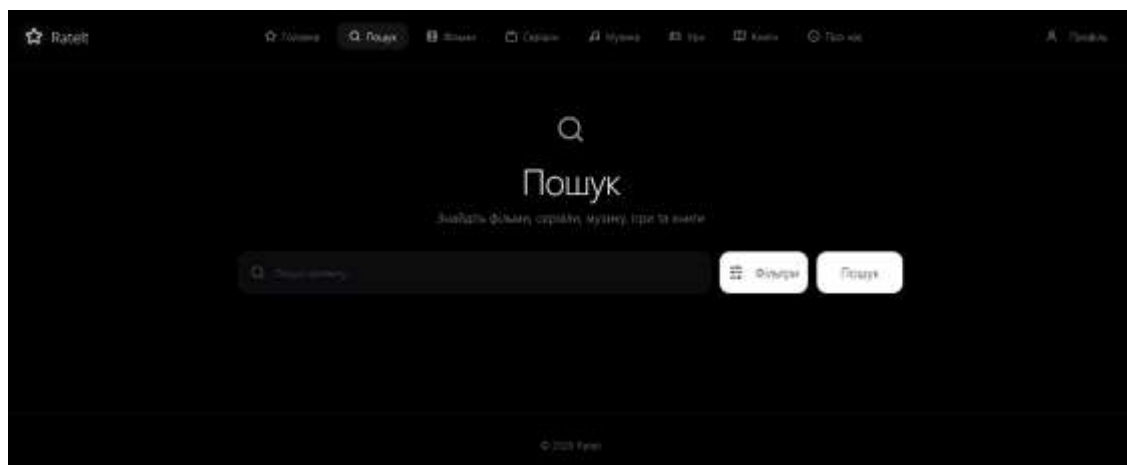


Рисунок 4.4 – Сторінка пошуку контенту

Для уточнення результатів пошуку користувач може скористатися системою фільтрів (див. рис. 4.5). Для цього необхідно натиснути кнопку «Фільтри», після чого відкривається меню з доступними параметрами фільтрації. Користувач може налаштувати такі фільтри:

- тип контенту (фільм, серіал, музика, гра, книга);
- діапазон року випуску (від певного року до певного року);
- мінімальний рейтинг контенту;
- сортування за роком виходу або за оцінкою;
- напрямок сортування (за зростанням або спаданням);
- жанр контенту.

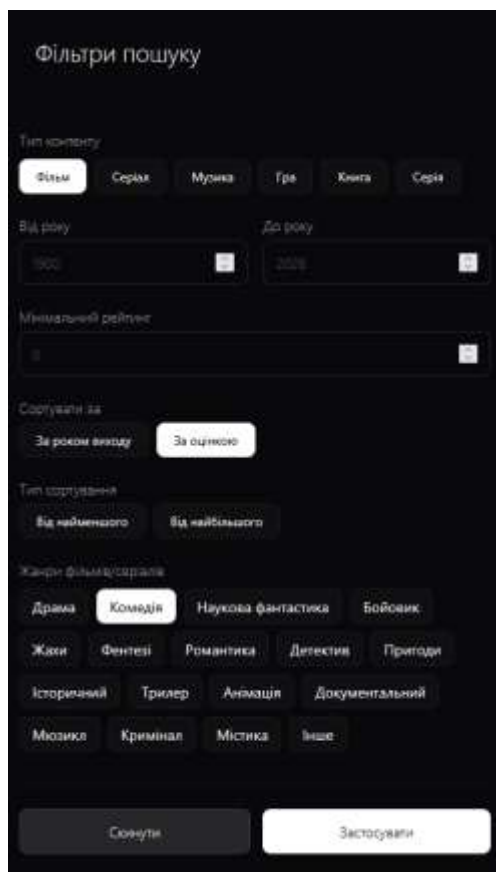


Рисунок 4.5 – Меню фільтрів пошуку

Після вибору необхідних параметрів користувач натискає кнопку «Застосувати», після чого клієнтська частина формує запит до API з урахуванням обраних фільтрів. На сторінці відображаються результати пошуку, що відповідають заданим критеріям (див. рис. 4.6).

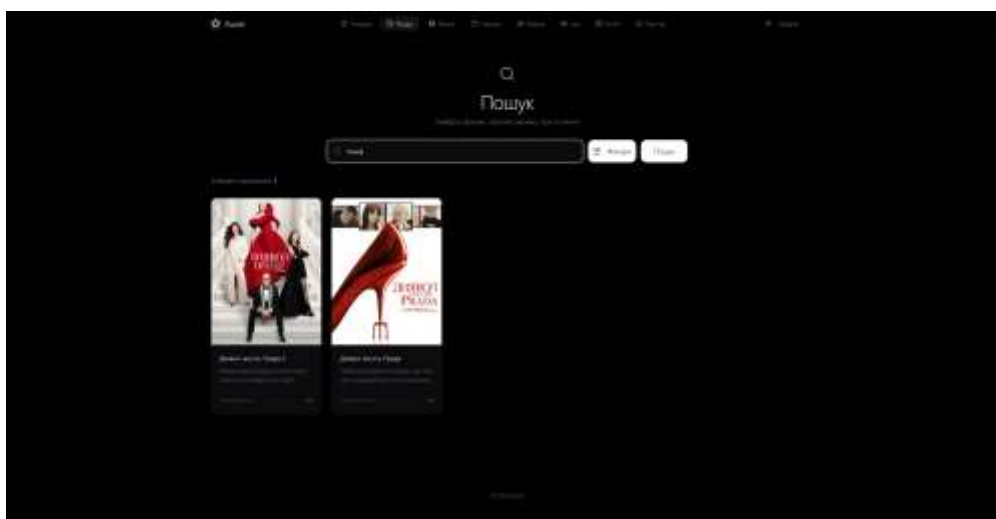


Рисунок 4.6 – Результати пошуку контенту

Для перегляду детальної інформації користувач натискає на картку відповідного контенту, після чого відкривається окрема сторінка з повним описом (див. рис. 4.7). Перелік відображуваних даних залежить від типу контенту та може включати:

- авторів контенту;
- видавця або студію;
- дату виходу;
- платформи розповсюдження;
- жанр або жанри;
- країну виробництва;
- тривалість;
- бюджет та касові збори;
- акторський склад та сценаристів.

Крім того, на сторінці контенту відображається середня оцінка платформи та відгуки користувачів.

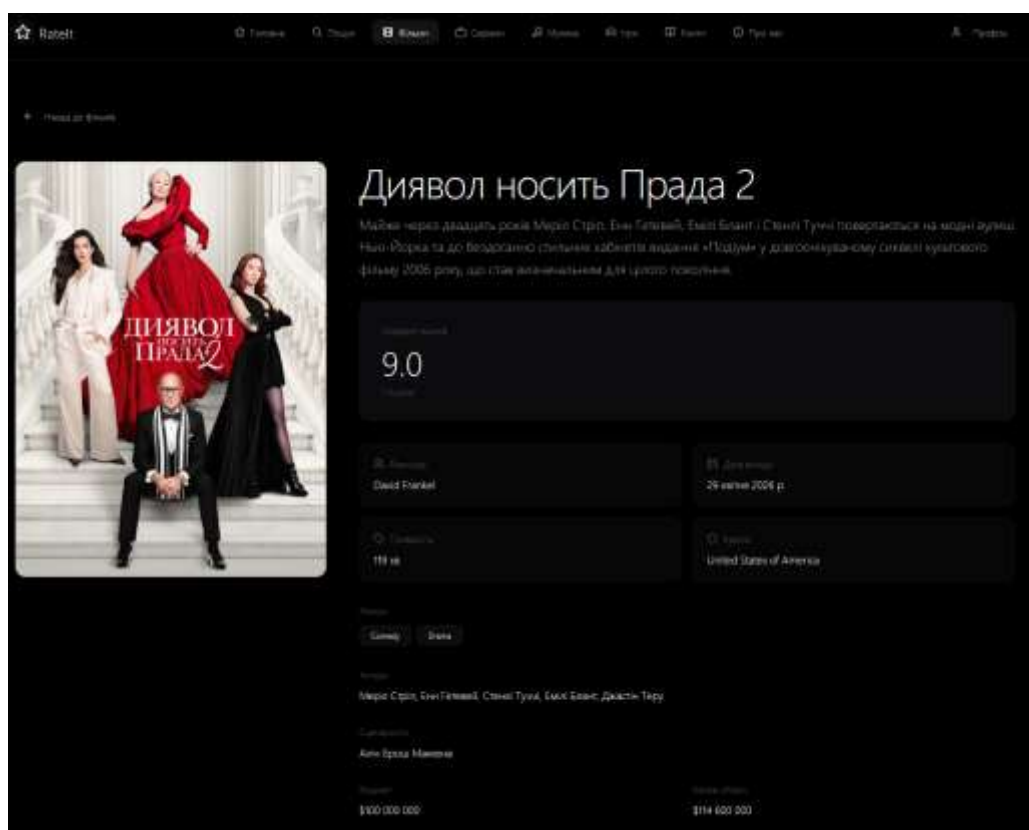


Рисунок 4.7 – Сторінка з детальною інформацією про контент

4.3 Залишення оцінки та відгуків про контент

На сторінці кожного контенту розміщено розділ із відгуками користувачів (див. рис. 4.8). У заголовку цього розділу відображається загальна кількість відгуків. Кожен відгук містить таку інформацію:

- ім'я користувача, який залишив відгук;
- оцінку контенту у форматі від 0 до 10;
- дату публікації відгуку;
- заголовок відгуку;
- текст відгуку;
- кількість лайків та дизлайків.



Рисунок 4.8 – Розділ відгуків на сторінці контенту

Залишати відгуки можуть лише авторизовані користувачі. Форма створення відгуку (див. рис. 4.9) розміщена безпосередньо під списком існуючих відгуків і містить такі поля:

- оцінка контенту за десятибальною шкалою, яка виставляється за допомогою зірочок;
- заголовок відгуку з підказкою для користувача;
- текст відгуку з підказкою «Поділіться своїми враженнями детальніше».

Після заповнення всіх обов'язкових полів користувач натискає кнопку «Опублікувати відгук», після чого клієнтська частина надсилає POST-запит до серверного API з даними відгуку у форматі JSON. Після успішного збереження відгук одразу відображається у розділі відгуків (див. рис. 4.10), а середня оцінка контенту автоматично перераховується з урахуванням нових даних.

Напишіть свій відгук

Ваша оцінка *

★★★★★★★★☆☆ 9/10

Заголовок відгуку *

Неймовірний фільм!

Ваш відгук *

Чесно, наткнувся на цей фільм випадково. Оцінювання були нульові. Після перегляду зрозумів що не знав про цей фільм раніше. Рекомендую.

Опублікувати відгук

Рисунок 4.9 – Форма створення відгуку

Відгуки (1)

Oleksandr Nermolenko 9/10
1 червня 2023 р.

Неймовірний фільм!

Чесно, наткнувся на цей фільм випадково. Оцінювання були нульові. Після перегляду зрозумів що не знав про цей фільм раніше. Рекомендую.

👍 1 🗨 0

Напишіть свій відгук

Ваша оцінка *

☆☆☆☆☆☆☆☆

Заголовок відгуку *

наприклад: неймовірний фільм

Ваш відгук *

Поділіться своїми враженнями детальноше.

Опублікувати відгук

Рисунок 4.10 – Створений відгук

Окрім перегляду відгуків на сторінках контенту, авторизований користувач має доступ до персональної сторінки «Мої відгуки», де зібрані всі відгуки, які він залишав на платформі (див. рис. 4.11).

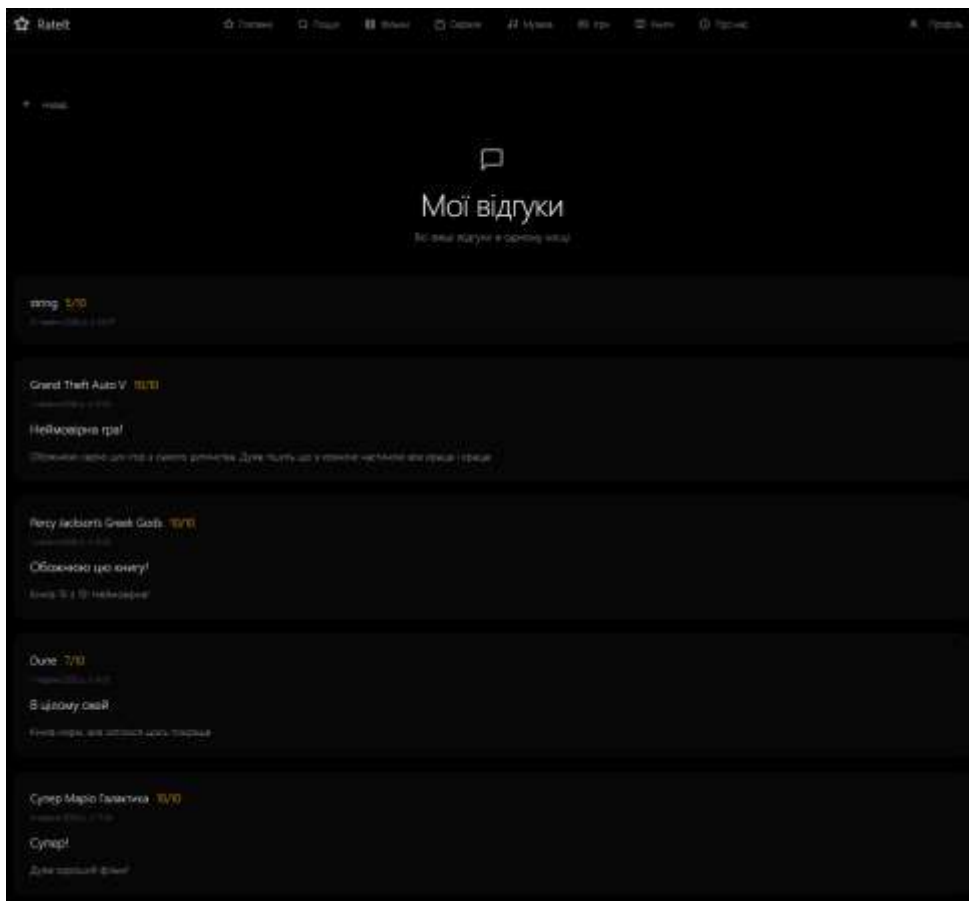


Рисунок 4.11 – Сторінка «Мої відгуки»

Ця сторінка забезпечує зручний перегляд та відстеження власної активності користувача, а також дозволяє переглядати історію всіх раніше опублікованих відгуків в одному місці.

4.4 Реалізація додавання, редагування та видалення контенту

Управління контентом платформи доступне лише користувачам із правами модератора. При завантаженні профілю клієнтська частина надсилає запит до серверного API для перевірки рівня прав поточного користувача. У разі підтвердження прав модератора у меню профілю з'являється додатковий розділ - «Панель модератора» (див. рис. 4.12).

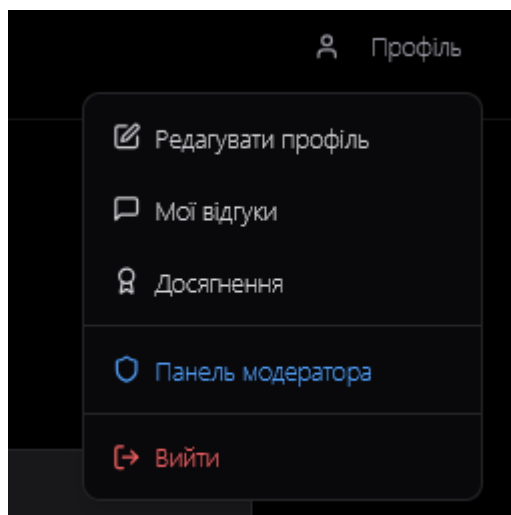


Рисунок 4.12 – Меню профілю з розділом панелі модератора

Панель модератора є окремою сторінкою веб-застосунку, що надає доступ до інструментів керування контентом платформи. У користувача є можливість додавання контенту. Інтерфейс панелі містить блоки додавання контенту та статистики (див. рис. 4.13).

Розділ «Додавання контенту» надає модератору можливість додавати новий контент до платформи. Для цього необхідно обрати тип контенту, натиснувши на відповідну кнопку: «Фільм», «Серіал», «Музика», «Гра» або «Книга». Кожен тип контенту позначено унікальною піктограмою для зручної візуальної навігації.

Розділ «Статистика платформи» відображає загальну кількість одиниць контенту, розміщеного на платформі, що дозволяє модератору отримувати узагальнену інформацію про наповнення системи.

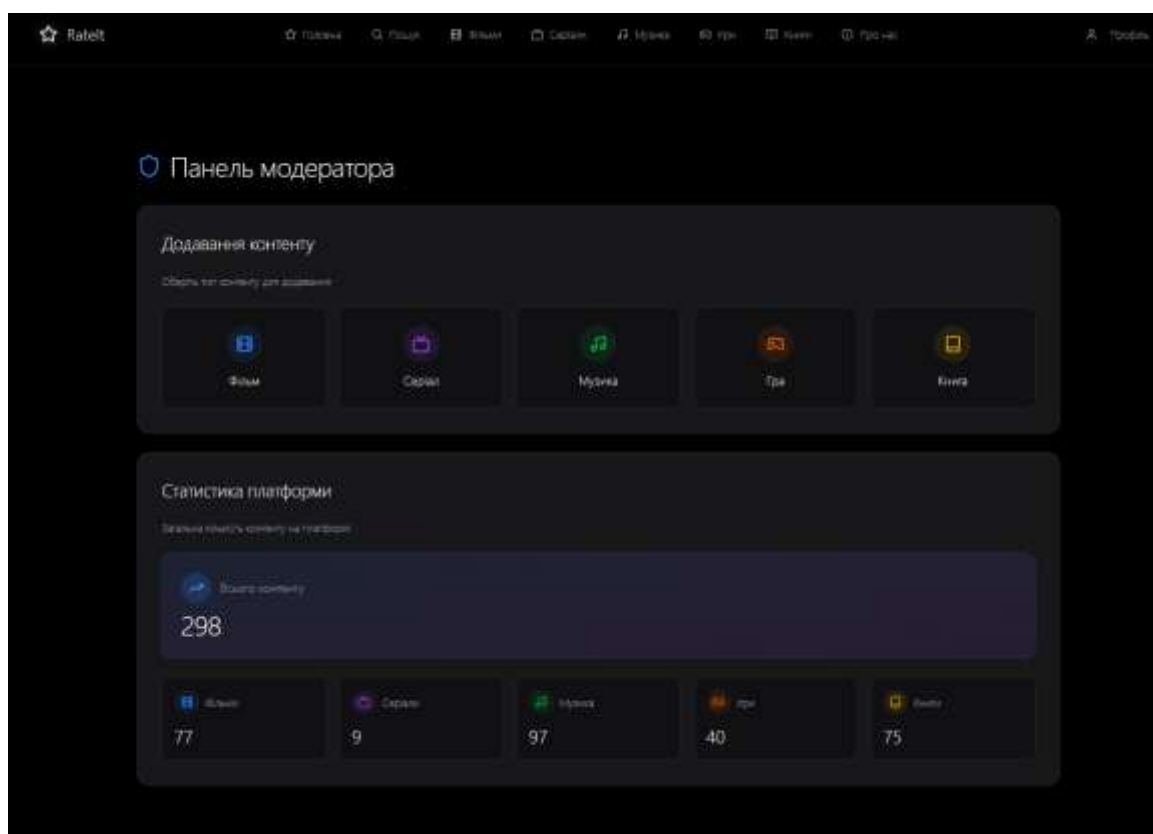


Рисунок 4.13 – Панель модератора

4.4.1 Додавання контенту

Для додавання нового контенту модератор переходить до відповідного розділу панелі та обирає тип контенту, який необхідно додати до системи. Залежно від обраного типу форма надає відповідний набір полів для заповнення. Наприклад, для фільму відображаються поля для введення назви, дати виходу, країни виробництва, жанру, тривалості, бюджету, касових зборів, акторського складу та сценаристів (див. рис. 4.14), тоді як для музичного альбому набір полів відрізнятиметься відповідно до специфіки цього типу контенту (див. рис. 4.15).

Після заповнення всіх необхідних полів модератор підтверджує створення, після чого клієнтська частина надсилає POST-запит до серверного API. У разі успішного збереження новий контент стає доступним для перегляду всіма користувачами платформи.

Додати фільм

Назва *

Опис *

Режисер *

Сценарист *

Рік випуску *

Рік завершення *

Тривалість (хв) *

Жанри *

Назва автора *

Додати фільм

Рисунок 4.14 – Форма додавання нового фільму

Додати музику

Назва *

Опис *

Артист *

Альбом *

Рік випуску *

Рік завершення *

Тривалість (хв) *

Жанри *

Назва автора *

Додати музику

Рисунок 4.15 – Форма додавання нової музики

4.4.2 Редагування контенту

Для редагування існуючого контенту модератор знаходить необхідний запис за допомогою пошуку та відкриває його сторінку. На сторінці контенту для модератора додатково відображаються кнопки «Редагувати» та «Видалити контент» (див. рис. 4.16). Після натискання кнопки «Редагувати» відкривається форма редагування з попередньо заповненими полями, що містить актуальні дані про контент (див. рис. 4.17). Модератор вносить необхідні зміни та натискає кнопку «Зберегти зміни», після чого клієнтська частина надсилає PUT-запит до серверного API з оновленими даними.

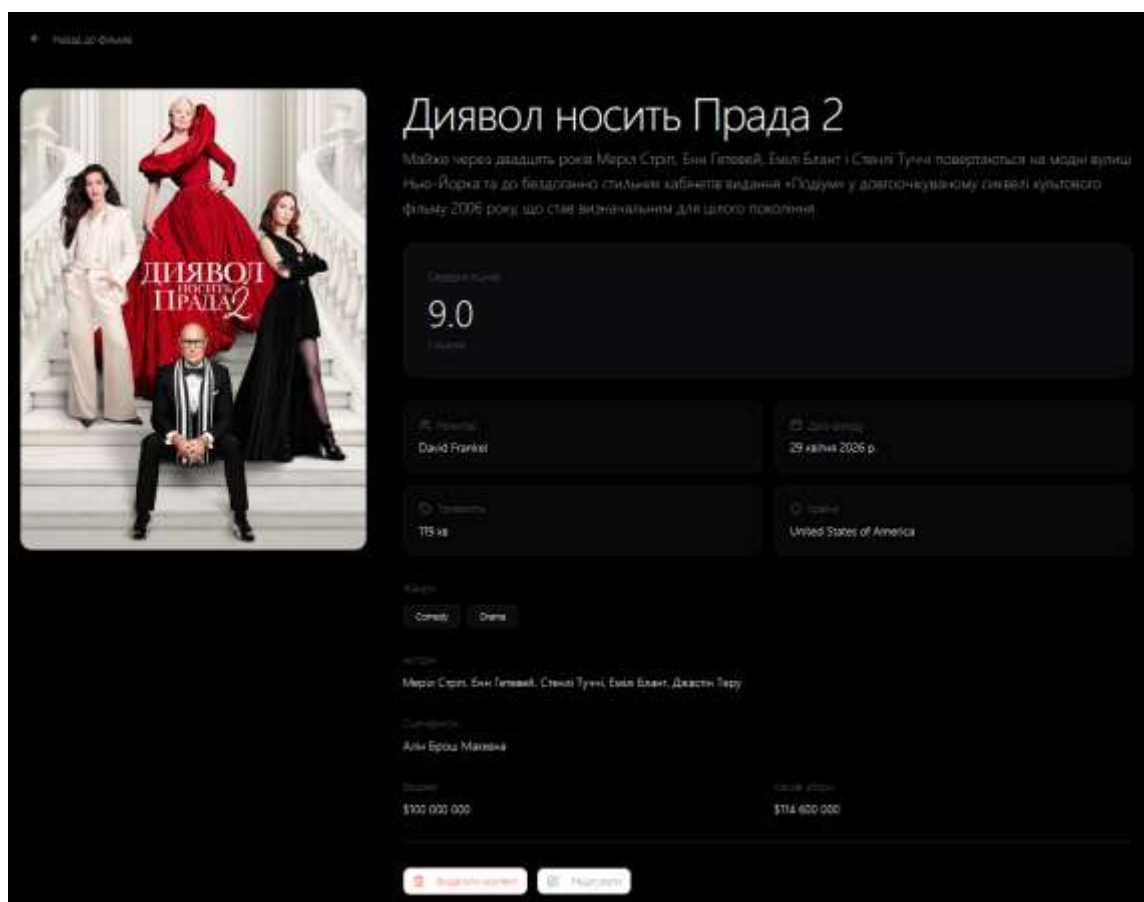


Рисунок 4.16 – Сторінка контенту з кнопками керування для модератора

Редагувати фільм

Назва *

Діалог ноль: Прада 2

Синopsis *

Майже через двадцять років Меріл Стріп, Енн Гетевей, Емілі Блант і Стенлі Туччі повертаються на модні вулиці Нью-Йорка та до бездоганно стильних кабінетів видання «Подум» у драгоцінному сніжці культового фільму 2006 року, що став визначальним для цього покоління.

Режисер *

David Frankel

Сценарист *

Alex Sposi, Maxima

Головні актори *

Meryl Streep, Ewan McGregor, Stanley Tucci, Emily Blunt, Dustin Tarr

Дата виходу *

29.04.2026

Рік виходу *

2026

Тривалість (хв)

119

Країна *

United States of America

Бюджет (\$)

100000000

Касові збори (\$)

114600000

Жанри *

☒ Drama ☒ Comedy ☐ Sci-Fi ☐ Action ☐ Music ☐ TV-Movie ☐ War ☐ Horror ☐ Fantasy ☐ Romance ☐ Detective ☐ Adventure ☐ Historical ☐ Thriller ☐ Musical ☐ Animation ☐ Documentary ☐ Family ☐ Western ☐ Crime ☐ Mystery

Настроювання *

Натюрморт

URL зображення *

<https://image.tmdb.org/t/p/w500/xiRCThaRSizukDTb00qKTDjd9v.jpg>

Рисунок 4.17 – Форма редагування контенту

4.4.3 Видалення контенту

Для видалення контенту модератор натискає кнопку «Видалити контент» на сторінці відповідного запису. З метою запобігання випадковому видаленню система відображає вікно, у якому модератор має підтвердити свій намір виконати цю дію перед остаточним видаленням. Після підтвердження клієнтська частина надсилає DELETE-запит до серверного API, і контент видаляється з платформи (див. рис. 4.18).

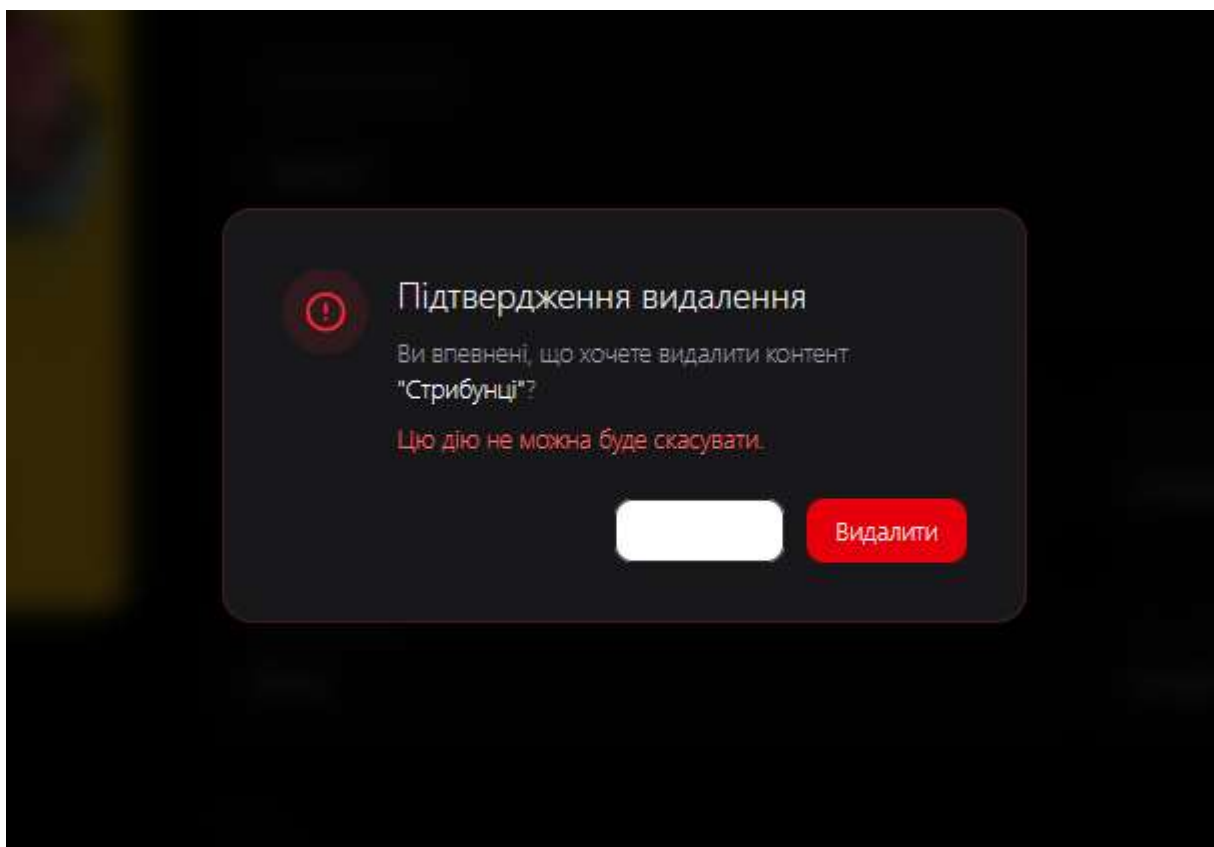


Рисунок 4.18 – Вікно підтвердження видалення контенту

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

У ході виконання курсового проєкту було повністю реалізовано клієнтську частину веб-застосунку RateIt, що забезпечує можливість перегляду, оцінювання та рецензування мультимедійного контенту різних категорій.

Розроблена система відповідає поставленим вимогам:

- реалізовано реєстрацію та автентифікацію користувачів із збереженням JWT-токена у localStorage;
- розроблено інтерфейс перегляду та пошуку контенту з гнучкою системою фільтрації за типом, жанром, роком випуску, рейтингом та параметрами сортування;
- реалізовано сторінки детального перегляду контенту з відображенням повної інформації залежно від його типу;
- впроваджено систему оцінювання та написання відгуків із автоматичним перерахунком середньої оцінки контенту;
- створено панель модератора з можливістю додавання, редагування та видалення контенту;
- забезпечено взаємодію клієнтської частини із серверним API через HTTP-запити з використанням JWT-токена для авторизації.

Інтерфейс веб-застосунку є інтуїтивно зрозумілим, зручним у використанні та адаптованим для роботи з різними категоріями мультимедійного контенту.

Висновки для подальшого розвитку:

- доцільним є впровадження персоналізованої системи рекомендацій контенту на основі вподобань користувача;
- перспективним напрямком є розширення соціального функціоналу, зокрема можливість підписки на інших користувачів та перегляду їхньої активності;
- оптимізація продуктивності клієнтської частини дозволить скоротити час завантаження великих обсягів контенту;
- можливим є додавання багатомовного інтерфейсу для розширення аудиторії платформи.

6 ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

6.1 Тестування веб-застосунку RateIt

Для перевірки працездатності веб-застосунку RateIt було проведено мануальне тестування всіх основних функціональних і нефункціональних компонентів системи.

Функціональне тестування включало:

- перевірку процесу реєстрації користувачів із заповненням обов'язкових полів (ім'я, прізвище, вік, електронна пошта, пароль) та коректності перенаправлення на сторінку входу після успішної реєстрації;
- тестування механізму автентифікації користувачів, перевірку збереження JWT-токена у localStorage та коректності авторизації подальших запитів до API;
- перевірку роботи системи пошуку контенту та фільтрації за типом контенту, діапазоном року випуску, мінімальним рейтингом та жанром;
- тестування сторінок детального перегляду контенту з перевіркою коректності відображення інформації залежно від типу медіаконтенту;
- перевірку роботи форми створення відгуків, виставлення оцінок за десятибальною шкалою та автоматичного перерахунку середньої оцінки контенту;
- тестування функціоналу лайків та дизлайків під відгуками користувачів із перевіркою коректного оновлення лічильників;
- перевірку роботи сторінки «Мої відгуки» та коректності відображення всіх раніше залишених користувачем відгуків;
- тестування панелі модератора, включаючи перевірку механізму розмежування прав доступу, а також функцій додавання, редагування та видалення контенту для кожної категорії.

Нефункціональне тестування охоплювало:

- перевірку стабільності роботи веб-застосунку під час одночасного використання різних модулів системи;
- перевірку сумісності із сучасними браузерами (Chrome, Firefox, Edge) та коректності відображення користувацького інтерфейсу;

- оцінку зручності користувацького інтерфейсу та інтуїтивності навігації між сторінками застосунку;

- перевірку коректності обмеження доступу до функціоналу залежно від рівня прав користувача.

Під час тестування активно використовувались інструменти відладки браузера та консоль розробника для контролю стану HTTP-запитів, відповідей серверного API та збереження даних у localStorage.

Результати:

- усі основні функції веб-застосунку працюють коректно та відповідають поставленим функціональним вимогам;

- критичних збоїв у роботі клієнтської частини після виправлення виявлених помилок не зафіксовано;

- незначні недоліки у відображенні окремих елементів інтерфейсу були оперативно усунені під час налагодження системи.

6.2 Приклади критичних та незначних помилок

Під час тестування було виявлено кілька помилок, які впливали на роботу веб-застосунку.

Найбільш критичною виявилась помилка у модулі автентифікації: після успішного входу в систему JWT-токен не зберігався у localStorage браузера. Внаслідок цього при кожному оновленні сторінки користувач автоматично виходив із системи, що унеможливлювало повноцінну роботу із застосунком. Після аналізу програмного коду було встановлено, що виклик localStorage.setItem() для збереження токена був відсутній в обробнику відповіді сервера. Помилку було усунено шляхом додавання відповідного виклику, після чого токен почав коректно зберігатися та використовуватися для авторизації запитів до API.

Також під час тестування панелі модератора було виявлено, що форма редагування контенту у деяких випадках не заповнювалась попередніми даними контенту. Проблема було вирішено шляхом коректного підвантаження даних з API перед відкриттям форми редагування.

Ще однією незначною помилкою було некоректне відображення середньої оцінки контенту одразу після публікації нового відгуку, бо значення оновлювалось лише після перезавантаження сторінки. Помилку усунено шляхом додавання локального оновлення стану компонента після успішної відповіді від сервера.

ВИСНОВКИ

У результаті виконання комплексного курсового проєкту було розроблено клієнтську частину веб-застосунку RateIt для оцінювання та рецензування мультимедійного контенту. Актуальність розробки зумовлена стрімким зростанням обсягів цифрового контенту та потребою користувачів у зручних інструментах для його пошуку, оцінювання та обміну власними враженнями.

Під час виконання роботи було проаналізовано сучасні веб-платформи для оцінювання мультимедійного контенту, визначено їхні переваги та недоліки, а також сформовано основні функціональні вимоги до майбутнього веб-застосунку. На основі проведеного аналізу було спроектовано структуру клієнтської частини системи та реалізовано сучасний користувацький інтерфейс.

Використання сучасного стеку технологій: React, TypeScript, Vite, HTML5 та CSS3 - забезпечило створення швидкого, адаптивного та інтуїтивно зрозумілого користувацького інтерфейсу, що взаємодіє із серверною частиною через API з використанням JWT-автентифікації.

У ході розробки було успішно реалізовано повний спектр запланованого функціоналу: систему реєстрації та автентифікації користувачів із зберіганням токена; пошуку контенту з гнучкою фільтрацією за типом, жанром, роком випуску, рейтингом і параметрами сортування; функціонал виставлення оцінок та написання рецензій із можливістю оцінювати відгуки інших користувачів; систему досягнень, що заохочує активне користування платформою; а також панель модератора з повним циклом керування контентом: додаванням, редагуванням та видаленням записів.

Під час тестування було перевірено працездатність основних функціональних і нефункціональних компонентів системи. Виявлені помилки були успішно усунуті, що дозволило забезпечити стабільну та коректну роботу веб-застосунку.

Розроблений веб-застосунок RateIt може бути використаний як основа для подальшого розвитку повноцінної платформи для оцінювання та рецензування мультимедійного контенту з додаванням соціальних функцій, персоналізованої

системи підбору контенту, підтримки багатомовності та створення власних списків контенту.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Тулашвілі Юрій Ю. Web-програмування. MERN fullstack development : навч. посіб. Луцьк : ЛНТУ, 2023. 242 с.
2. Bugl D. Learn react hooks: build and refactor modern react. js applications using hooks. Packt Publishing, Limited, 2019. 426 с.
3. *React*. URL: <http://react.dev/learn> (дата звернення: 09.04.2026).
4. Romanyshyn Y., Shtykh A. Information content of media internet-resources: analytic review. *Scientific journal "library science. record studies. informology"*. 2023. № 3. URL: <https://doi.org/10.32461/2409-9805.3.2023.290987> (дата звернення: 14.05.2026).
5. Web-програмування. Частина 1 (frontend) : навч. посіб. Кропивницький : ЦНТУ, 2022. 208 с.
6. Wieruch R. The road to react: your journey to master plain yet pragmatic react.js. Independently published, 2018. 226 с.
7. GitHub. URL: https://github.com/NureIermolenkoOleksandr/2026_B_KKP_PZPI-23-4_Yermolenko_O_S.

ДОДАТОК А

Звіт результатів перевірки на унікальність тексту в базі ХНУРЕ

Звіт не був оцінений

Метадані

ДОКУМЕНТ

Заголовок

2026_Б_ККП_ПЗПІ-23_4_Єрмоленко_О_С_скорочений

Автор

Науковий керівник / Експерт

Єрмоленко О.С.

Світлана Нестерцова

ІД документу

334422431

ОРГАНІЗАЦІЯ

Назва організації

Kharkiv National University of Radio Electronics

підрозділ

каф. ПІ

ЗВІТ

Дата звіту

2026-06-25

Дата редагування

Обсяг знайдених подібностей

Коефіцієнт подібності визначає, який відсоток тексту по відношенню до загального обсягу тексту було знайдено в різних джерелах. Заверніть увагу, що високі значення коефіцієнта не автоматично означають плагіат. Звіт має аналізувати компетентна / уповноважена особа.

2.92%

2.92%

КП 1

4242

Кількість слів

0.87%

0.87%

КЦ

34948

Кількість символів

Тривога

У цьому розділі ви знайдете інформацію щодо текстових спотворень. Ці спотворення в тексті можуть говорити про МОЖЛИВІ маніпуляції в тексті. Спотворення в тексті можуть мати навмисний характер, але частіше характер технічних помилок при конвертації документа та його збереженні, тому ми рекомендуємо вам підходити до аналізу цього модуля відповідально. У разі виникнення запитань, просимо звертатися до нашої служби підтримки.

Заміна букв		0
Інтервали		0
Мікропробіли		0
Білі знаки		0
Парафрази (SmartMarks)		7

Джерела

Нижче наведений список джерел. В цьому списку є джерела із різних баз даних. Колір тексту означає в якому джерелі він був знайдений. Ці джерела і значення Коефіцієнту Подібності не відображають прямого плагіату. Необхідно відкрити кожне джерело і проаналізувати зміст і правильність оформлення джерела.

10 найдовших фраз

Копіювати текст

#	НАЗВА ТА АДРЕСА ДЖЕРЕЛА URL (НАЗВА БАЗИ)	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	https://openarchive.nure.ua/bitstreams/2231727c-4bb1-4e3d-a26a-6e7e962f1c73/download	20 0 (0.47 %)

2	https://openarchive.nure.ua/bitstreams/2231727c-4bb1-4e3d-a26a-6e7e962f1c73/download	15 0 (0.35 %)
3	https://openarchive.nure.ua/bitstreams/2231727c-4bb1-4e3d-a26a-6e7e962f1c73/download	14 0 (0.33 %)
4	https://openarchive.nure.ua/bitstreams/9af66b19-61bd-4b82-9767-10bc363c26f8/download	14 0 (0.33 %)
5	https://openarchive.nure.ua/bitstreams/2231727c-4bb1-4e3d-a26a-6e7e962f1c73/download	10 0 (0.24 %)
6	https://openarchive.nure.ua/server/api/core/bitstreams/ad9f1307-d66e-4247-a961-9cc9659ea364/content	10 0 (0.24 %)
7	Бакалавр_КП-22_Саврацький_Євгеній_Вікторович 5/29/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФПСПМ, К-ра програмного забезпечення комп'ютерних систем)	9 0 (0.21 %)
8	https://openarchive.nure.ua/bitstreams/2231727c-4bb1-4e3d-a26a-6e7e962f1c73/download	7 0 (0.17 %)
9	Бакалавр_КП-22_Саврацький_Євгеній_Вікторович 5/29/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФПСПМ, К-ра програмного забезпечення комп'ютерних систем)	7 0 (0.17 %)
10	2024_M_IMC_KP_ІПЗм-23-1_Лалін_В_І_Трофіменко_О_О_скорочений 6/4/2024 Kharkiv National University of Radio Electronics (Харківський національний університет радіоелектроніки)	7 0 (0.17 %)
База даних RefBooks		
#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
Домашня база даних (0.17 %)		
#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	2024_M_IMC_KP_ІПЗм-23-1_Лалін_В_І_Трофіменко_О_О_скорочений 6/4/2024 Kharkiv National University of Radio Electronics (Харківський національний університет радіоелектроніки)	7 (1) (0.17 %)
Програма обміну базами даних (0.38 %)		
#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
2	Бакалавр_КП-22_Саврацький_Євгеній_Вікторович 5/29/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФПСПМ, К-ра програмного забезпечення комп'ютерних систем)	16 (2) (0.38 %)
Інтернет (2.38 %)		
#	ДЖЕРЕЛО URL	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
	https://openarchive.nure.ua/bitstreams/2231727c-4bb1-4e3d-a26a-6e7e962f1c73/download	66 (15) (4.66 %)

3	https://openarchive.nure.ua/bitstreams/66217675-7534-4698-991e29611570/download	90 (1) (1.00 %)
4	https://openarchive.nure.ua/bitstreams/9af66b19-61bd-4b82-9767-10bc363c26f8/download	14 (1) (0.33 %)
5	https://openarchive.nure.ua/server/api/core/bitstreams/ad9f1307-d66e-4247-a961-9ce9659ea364/content	10 (1) (0.24 %)
6	https://openarchive.nure.ua/bitstreams/43e9a84a-7475-470b-b8e8-4d474fff7c81/download	6 (1) (0.14 %)
7	https://studfile.net/preview/3021338/	5 (1) (0.12 %)



Список прийнятих фрагментів

#	ЗМІСТ	КІЛЬКІСТЬ ОДНАКОВИХ СЛІВ (ФРАГМЕНТІВ)
---	-------	---------------------------------------

ДОДАТОК Б

Слайди презентації

МІНІСТЕРСТВО
ОСВІТИ І НАУКИ
УКРАЇНИ

ХАРКІВСЬКИЙ
НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ

Веб-застосунок для
оцінювання та рецензування
мультимедійного контенту

Виконав:
ст. гр. ПЗПІ-23-4
Єрмоленко О. С.

Науковий керівник:
доц. каф. ПІ
Чуприна А. С.

SE
software
engineering

Мета роботи

Метою курсової роботи є розробка клієнтської частини веб-застосунку для оцінювання та рецензування мультимедійного контенту з використанням React, TypeScript та [Vite](#).

Стрімке зростання обсягів цифрового контенту формує потребу в зручних інструментах для його оцінювання. Більшість існуючих платформ орієнтовані лише на один тип медіа, що змушує користувачів використовувати декілька різних сервісів одночасно. [Ratelt](#) вирішує цю проблему, об'єднуючи всі категорії контенту в єдиному інтерфейсі.

Аналіз існуючих рішень



Лише фільми та серіали, відсутні інші категорії медіа



Letterboxd

Виключно кіно контент, немає музики, ігор та книг



metacritic

Охоплює кілька категорій, але складний інтерфейс



3

Постановка задачі та опис системи

Формулювання проблеми:

- відсутня єдина платформа для оцінювання контенту різних категорій;
- більшість аналогів мають застарілий дизайн та обмежений функціонал.

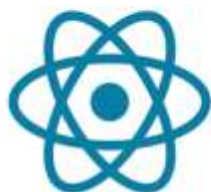
Очікувані результати:

- єдиний інтерфейс для всіх категорій медіа;
- реєстрація, автентифікація, пошук, фільтрація, відгуки та панель модератора.



4

Вибір технологій розробки



Бібліотека для
побудови інтерфейсу



Інструмент збірки
проекту



Типізована надбудова
над JavaScript



Середовище розробки

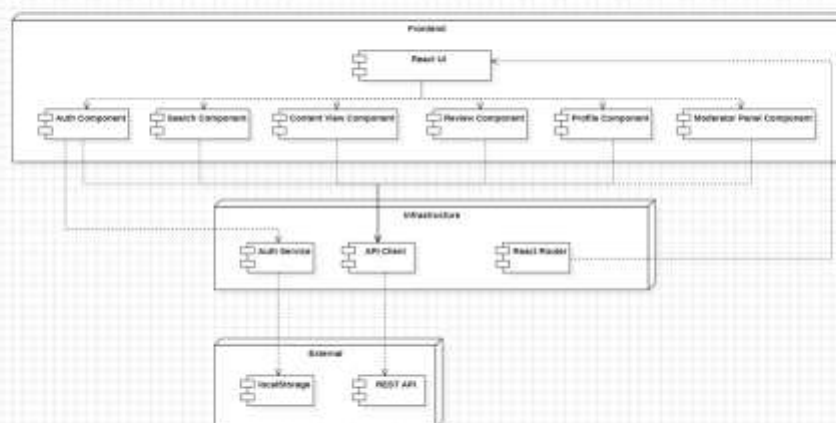


Розмітка та стилізація
інтерфейсу



5

Архітектура створеного програмного забезпечення



6

Опис програмного забезпечення, що було використано у дослідженні

- React - бібліотека для побудови користувацького інтерфейсу на основі компонентного підходу, що дозволяє повторно використовувати елементи інтерфейсу.
- TypeScript - типізована надбудова над JavaScript, що забезпечує виявлення помилок на етапі розробки та підвищує надійність коду.
- Vite - сучасний інструмент збірки фронтенд-проектів із миттєвим запуском середовища розробки та оптимізованою продакшн-збіркою.
- HTML5 / CSS3 - стандартні технології для розмітки та стилізації користувацького інтерфейсу веб-застосунку.
- Visual Studio Code - середовище розробки з підтримкою TypeScript, розширеннями для React та інструментами налагодження.



Дизайн системи

Методи: клієнт-серверна архітектура, компонентний підхід React

Послідовність: Реєстрація → Авторизація → Пошук контенту → Перегляд деталей → Виставлення оцінки та написання відгуку

Технології: React, TypeScript, Vite, HTML5/CSS3, Visual Studio Code.



Приклад реалізації

```
private getAuthHeader(): HeadersInit {
  const token = localStorage.getItem("jwt_token");
  if (token) {
    return { Authorization: `Bearer ${token}` };
  }
  return {};
}
async get<T>(endpoint: string, requiresAuth = false):
Promise<T> {
  const headers: HeadersInit = {
    ...(requiresAuth ? this.getAuthHeader() : {}),
  };
  const response = await fetch(`${this.baseUrl}${endpoint}`, {
    method: "GET",
    headers,
  });
  if (!response.ok) throw new Error(`HTTP error! status:
  ${response.status}`);
  return response.json();
}
```



9

Інтерфейс користувача



Головна сторінка



Сторінка досягнень



Сторінка пошуку



Форма для написання відгуків



Сторінка з інформацією про контент



10

Підсумки

У роботі було розроблено клієнтську частину веб-застосунку Ratelt - універсальну платформу для оцінювання та рецензування мультимедійного контенту. Було реалізовано систему реєстрації та автентифікації користувачів із збереженням JWT-токена, зручний інтерфейс пошуку та фільтрації контенту за різними параметрами, функціонал виставлення оцінок і написання відгуків із можливістю лайків, а також панель модератора для керування контентом платформи.

ДОДАТОК В

Специфікація програмного забезпечення

Multimedia Content Rating and Evaluation Platform

Team Ratelt

Kulakov Serhii

Yermolenko Oleksandr

Software Requirements Specification

Document

Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Scope	5
1.3 Definitions, Acronyms, and Abbreviations	6
1.4 References	6
1.5 Overview	6
 2. The Overall Description	 7
2.1 Product Perspective	7
2.1.1 System Interfaces	7
2.1.2 Interfaces	8
2.1.3 Hardware Interfaces	9
2.1.4 Software Interfaces	10
2.1.5 Communications Interfaces	11
2.1.6 Memory Constraints	12
2.1.7 Operations	12
2.1.8 Site Adaptation Requirements	13
2.2 Product Functions	14
2.3 User Characteristics	15
2.4 Constraints	15
2.5 Assumptions and Dependencies	15
2.6 Apportioning of Requirements	16
 3. Specific Requirements	 17
3.1 External interfaces	17
3.2 Functions	18
3.3 Performance Requirements	19
3.4 Logical Database Requirements	20
3.5 Design Constraints	21
3.5.1 Standards Compliance	21
3.6 Software System Attributes	22
3.6.1 Reliability	22
3.6.2 Availability	22
3.6.3 Security	22
3.6.4 Maintainability	23
3.6.5 Portability	23
3.7 Organizing the Specific Requirements	23
3.7.1 System Mode	24
3.7.2 User Class	24
3.7.3 Objects	24
3.7.4 Feature	25
3.7.5 Stimulus	25

3.7.6 Response	26
3.7.7 Functional Hierarchy	26
4. Change Management Process	26
4.1 <i>General Provisions</i>	26
4.2 <i>Roles and Responsibilities</i>	26
4.3 <i>Submitting a Change Request</i>	27
4.4 <i>Evaluation and Decision-Making</i>	27
4.5 <i>Implementing Changes</i>	27
5. Document Approvals	27
6. Supporting Information	28

1 INTRODUCTION (ВСТУП)

1.1 Purpose (Призначення)

Метою цього документа є детальний опис функціональних та нефункціональних вимог до платформи “RateIt”. Цей документ призначено для розробників, тестувальників та менеджерів проекту. Основним призначенням системи є надання користувачам інструменту для об’єктивного оцінювання та обговорення різноманітного виду цифрового контенту (кіно, ігор, музики тощо) в межах єдиної екосистеми. Проект спрямований на вирішення проблеми надлишку інформації шляхом впровадження соціальних фільтрів та механік гейміфікації, які стимулюють створення якісного користувацького контенту.

1.2 Scope (Область дії)

Назва продукту: Програмним продуктом, що описіється в цій специфікації, є кросплатформена вебсистема “RateIt”.

Функціональні межі: Система призначена для агрегації та оцінювання мультимедійного контенту та формування користувацької спільноти навколо цифрового контенту

Система буде:

- надавати інструменти для оцінювання та рецензування мультимедійного контенту;
- впроваджувати механізми гейміфікації для стимулювання активності користувачів;
- забезпечувати соціальну взаємодію між користувачами.

Система не буде:

- надавати можливість безпосереднього перегляду/прослуховування/прочитання або завантаження піратського контенту (система є агрегатором думок, а не медіа-хостингом) та не буде функціонувати як інтернет-магазин для продажу товарів.

Застосування: Платформа використовуватиметься як соціальний навігатор у світі розважального контенту, де кожен користувач зможе знайти для себе цікавий контент

Цілі: мінімізувати час користувача на пошук якісного продукту через систему “розумних” соціальних фільтрів.

Переваги: користувач отримує об'єктивну оцінку від спільноти, а завдяки гейміфікації процес оцінювання та рецензування контенту стає інтерактивним.

Мета: створити єдину точку входу для оцінки будь-якого цифрового досвіду.

1.3 Definitions, acronyms & abbreviations (Визначення, акроніми та скорочення)

Таблиця 1 - Definitions (Визначення)

Термін	Визначення
Гейміфікація	Використання ігрових елементів та механік (бали, рівня, досягнення) у неігровому контексті для підвищення залученості користувачів
Агрегатор	Програмне забезпечення, що збирає, структурує та відображає інформацію про об'єкти з різних джерел в одному інтерфейсі
Рецензія	Розгорнутий текстовий відгук користувача, що містить аналіз та оцінку контенту
Мультимедійний контент	Сукупність цифрових продуктів різних типів, включаючи: фільми, серіали, відеоігри, музичні альбоми та книги

Таблиця 2 - Acronyms (Акроніми)

Акронім	Повне значення
MVP	Minimal Viable Product (Мінімально життєздатний продукт)
SRS	Software Requirements Specification (Специфікація вимог до програмного забезпечення)
API	Application Programming Interface (Програмний

	інтерфейс додатку)
UI	User Interface (Інтерфейс користувача)
UX	User Experience (Досвід користувача)

Таблиця 3 - Abbreviations (Скорочення)

Скорочення	Повне значення
БД	База даних
ПЗ	Програмне забезпечення
ТЗ	Технічне завдання
ІТ	Інформаційні технології

1.4 References (Посилання)

1. IEEE Std 830-1998 (IEEE Recommended Practice for Software Requirements Specifications): <https://ieeexplore.ieee.org>
2. IEEE-Standart-SRS-Template: <https://dl.nure.ua/mod/resource/view.php?id=804215>

1.5 Overview (Огляд)

Цей документ (SRS) структурований таким чином, щоб забезпечити швидкий доступ до необхідної інформації для різних груп спеціалістів:

Розділ 2: Overall Description (Загальний опис) містить огляд головних функцій платформи “RateIt”, опис груп користувачів та припущень щодо розробки.

Розділ 3: Specific Requirements (Функціональні вимоги) деталізує конкретні сценарії взаємодії користувача з системою. Цей розділ є ключовим для розробників та тестувальників, оскільки містить логіку кожного модуля: від системи оцінювання до механік гейміфікації.

2 THE OVERALL DESCRIPTION (ЗАГАЛЬНИЙ ОПИС)

Платформа “RateIt” розробляється як відповідь на проблему інформаційного перенавантаження. Сьогодні користувачеві важко обрати якісний фільм, гру чи книгу. Наша платформа ставить собі на меті стати соціальним навігатором та місцем для поціновувачів контенту, які бажають ділитися своїми думками з іншими людьми. Головна ідея продукту полягає у створенні інтелектуальної системи соціальної фільтрації, де пріоритет надається живим та реальним відгукам людей. Щоб зробити процес оцінювання цікавішим, у платформу будуть інтегровані

механіки гейміфікації: за свою активність користувачі отримуватимуть унікальні досягнення та підвищуватимуть свій рівень експертності. Це перетворить просте виставлення оцінок на цікавий процес, що стимулюватиме користувачів на написання якісних та змістовних відгуків.

2.1 Product perspective (Перспективи продукту)

Платформа “RateIt” розробляється як самостійний та незалежний програмний продукт, який не є модулем чи компонентом сторонньої системи. На ринку існують платформи аналоги, але, на відміну від них - вузькоспеціалізованих сервісів, таких як IMDb чи Letterboxd (оцінювання кіно), Goodreads (оцінювання книг), RAWG (оцінювання ігор) та ін., наш продукт є універсальним хабом. Такий підхід дозволяє користувачу оцінювати різноманітний контент на єдиному сервісі.

Ключові відмінності:

- крос-платформеність контенту: об’єднання різних типів медіа на одній платформі;
- соціальна інтеграція: платформа робить акцент на створенні живої спільноти;
- гейміфікація: використання ігрових механік (система рівнів, досягнень та нагород) як інструмент заохочення активності та стимулювання на написання якісних рецензій.

Хоча “RateIt” є автономним застосунком, його функціонування передбачає інтеграцію з зовнішніми БД мультимедійного контенту через прикладні програмні інтерфейси (API). Це дозволяє підтримувати актуальність каталогу без ручного керування метаданими об’єктів.

2.1.1 System interfaces (Системні інтерфейси)

Для повноцінного функціонування платформа “RateIt” взаємодіє з наступними зовнішніми системами:

1. Зовнішні бази метаданих: система підключається до сторонніх API (таких як TMDb для фільмів, IGDB для відеігор, Google Books для книг тощо). Це дозволяє ПЗ автоматично імпортувати головну інформацію про медіаконтент (назва, дата виходу, опис і т.д.). Така взаємодія здійснюється через RESTful API за допомогою JSON-запитів.

2. Сервіси автентифікації: взаємодія із зовнішніми сервісами Google та Facebook, що дозволяє користувачам швидко реєструватися в системі без створення додаткового пароля, використовуючі вже існуючі облікові записи. Для реалізації такого функціоналу використовується протокол OAuth 2.0 для безпечної передачі токенів авторизації.

3. Система управління базами даних: взаємодія з реляційною БД для збереження структурованих даних та Redis для кешування. Програмне

забезпечення виконує операції запису та читання профілів користувачів, їхніх оцінок, рецензій та прогресу в системі гейміфікації.

4. Хмарні схвища мультимедіа: взаємодія з сервісами для зберігання та швидкої роздачі користувацького контенту (аватарок, завантажених зображень до рецензій).

2.1.2 Interfaces (Інтерфейси)

Взаємодія користувача з нашою платформою побудована на принципі інтуїтивності та швидкості доступу до інформації, що дозволяє зосередитись на контенті.

Логічні характеристики інтерфейсів:

- Тип інтерфейсу: основним інструментом взаємодії є графічний інтерфейс користувача (GUI), адаптований для сучасних веббраузерів. Система не потребує використання командного рядка або спеціальних терміналів.

- Структура взаємодії: навігація реалізована через модель, де користувач може переходити від глобального пошуку та категорій до детальних карток конкретного об'єкту фільму, книги, гри і т. д.

- Інформаційна архітектура: кожна сторінка має уніфіковану розмітку. Ключові елементи: кнопки оцінювання, блоки рецензії та кнопки соціальної взаємодії, які розташовані у звичних для користувача зонах видимості.

- Механізм введення даних: введення даних, наприклад вибір контенту, фільтрація, сортування, рецензування і т. п., виконується через інтерактивні елементи: зіркові шкали, слайдери та випадаючі списки, що мінімізує необхідність ручного введення тексту.

Оптимізація взаємодії з користувачем:

- Адаптивність: інтерфейс працює однаково ефективно як на десктопних моніторах так і на мобільних пристроях, автоматично змінюючи розміри елементів під розмір девайсу.

- Доступність: платформа враховує потреби людей з обмеженнями: передбачено високу контрастність тексту, зміну розміру шрифту, можливість керування з клавіатури та сумисність з програмами для читання екрана.

- Ефективність навігації: основні дії на платформі (пошук та оцінка контенту) доступні в один-два кліки з будь-якого розділу.

- Візуальний відгук: кожна дія (збереження відгуку, лайк на відгук іншого користувача) супроводжується візуальною анімацією.

2.1.3 Hardware Interfaces (Апаратні інтерфейси)

Оскільки платформа “RateIt” є вебзастосунком, вона не передбачає прямого керування спецефічними апаратними пристроями або взаємодіями з нестандартними обладнанням.

Основні положення:

- Прямі інтерфейси: система не має вимог до спецефічних апаратних інтерфейсів. ПЗ не здійснює прямий контроль над фізичними пристроями.
- Пристрої підтримки: взаємодія з апаратним ПЗ відбувається опосередковано через веббраузер та операційну систему. Платформа підтримує стандартні пристрої введення (клавіатура, миша, сенсорні екрани) та пристрої введення (монітори, дисплеї мобільних девайсів).
- Протоколи: вся апаратна комунікація делегується на рівень операційної системи та мережевих протоколів стандартних інтернет-браузерів.

2.1.4 Software interfaces (Програмні інтерфейси)

Функціонування платформи “RateIt” базується на інтеграції з набором зовнішніх програмних продуктів, які забезпечують стабільну роботу користувацького інтерфейсу.

2.1.4.1 Система управління базами даних (PostgreSQL/ MS SQL Server)

- Призначення: використовується як основне сховище для збереження основної інформації платформи (профілі користувачів, рецензії, об’єкти медіаконтенту тощо).

- Визначення інтерфейсу: взаємодія здійснюється через стандартні SQL-драйвери та ORM-системи. Система передає запити на CRUD операції та отримує результати у вигляді структурованих наборів даних.

2.1.4.2 Зовнішні сервіси метаданих (TMDb, IGDB, Google Books APIs)

- Призначення: використовуються для отримання актуальної інформації про всі види контенту, що підтримуються системою, для наповнення каталогу платформи.

- Визначення інтерфейсу: взаємодія відбувається через RESTful API. Обмін повідомленнями здійснюється за допомогою HTTP-запитів, а контент передається у JSON-форматі.

2.1.4.3 Середовище виконання бекенду

- Призначення: забезпечує логіку обробки запитів, валідацію даних та координацію між БД і клієнтською частиною.

- Визначення інтерфейсу: взаємодія відбувається через RESTful API. Обмін повідомленнями здійснюється за допомогою HTTP-запитів, а контент передається у JSON-форматі.

2.1.4.4 Клієнтське середовище (Браузери)

- Призначення: веббраузери користувачів виступають середовищем виконання для фронтенд-частини застосунку.

- Визначення інтерфейсу: взаємодія базується на стандартах HTML5, CSS3 та Javascript. Обмін даними з сервером відбувається через асинхронні запити.

2.1.5 Communication interfaces (Комунікаційні інтерфейси)

Для обміну даними між клієнтською частиною, сервером та зовнішніми серісами платформа “RateIt” використовує стандартизовані протоколи.

Основні інтерфейси зв'язку:

- Протокол передачі даних (HTTP/HTTPS): вся комунікація між веббраузером та сервером, між сервером та зовнішніми API здійснюється за протоколом HTTP. Посилання на стандарт: <https://datatracker.ietf.org/doc/html/rfc2818>

- Формат обміну повідомленнями (JSON): основним форматом передачі даних між бекендом та фронтендом є JSON. Цей формат використовується для всіх API-запитів та відповідей. Посилання на стандарт: <https://datatracker.ietf.org/doc/html/rfc8259>

- Метод взаємодії (REST): Система використовує архітектурний стиль REST для побудови програмних інтерфейсів, що передбачає використання стандартних HTTP-методів (GET, POST, PUT, DELETE) для роботи з ресурсами системи.

- Кодування символів (UTF-8): для забезпечення мультимовного інтерфейсу всі комунікаційні інтерфейси використовують кодування UTF-8.

2.1.6 Memory Constraints (Обмеження пам'яті)

Оскільки платформа “RateIt” функціонує в середовищі веббраузера та хмарного сервера, основні обмеження пам’яті стосуються обсягів зберігання даних у базі та кешування для швидкого доступу.

Основні характеристики та запити:

- Серверна оперативна пам’ять: для стабільної роботи серверної частини мінімально необхідний обсяг становить 512 MB RAM. Цього достатньо для обробки черги запитів та базового кешування об’єктів.

- Дисковий простір:

1. База даних: початковий обсяг сховища для текстових даних (профілі, рецензії, оцінки і т.п.) має становити не менше 1 GB з можливістю динамічного масштабування.

2. Мультимедіа: обмеження на завантаження медіафайлів встановлюються на рівні 2 MB на один файл для економії місця в хмарному сховищі.

- Клієнтська пам’ять: програмний код фронтенд частини оптимізовано таким чином, щоб споживання в браузері користувача не перевищувало 200 MB, що забезпечує стабільну роботу на пристрої середньої цінової категорії.

- Локальне сховище: для керуваннями сесій використовується не більше 5-10 MB.

2.1.7 Operations (Експлуатація)

Для забезпечення безперебійного доступу до сервісу “RateIt” та підтримки актуальності даних, система передбачає чітко визначені режими роботи та автоматизовані процедури обслуговування.

Режими роботи: система функціонує у двох режимах:

- Користувацький режим: функції для оцінювання та перегляду рецензій до контенту;

- Адміністративний режим: модерація контенту, управління скаргами тощо.

Періоди роботи:

- Інтерактивний період: система працює в режимі реального часу 24/7;

- Автономний період: виконання запланованих технічних робіт у години найнижчої активності користувачів, при цьому не обмежуючи доступ до основних функцій.

Функції підтримки обробки даних: система здійснює автоматичний збір та валідацію даних, які надходять від користувачів. Впроваджено фонові процеси для аналізу активності та оновлення статусів гіміфікації.

Резервне копіювання та відновлення: автоматичне копіювання БД створюється щоденно. Механізми відновлення повинні забезпечувати повернення системи до робочого стану з останньої збереженої точки у разі критичних збоїв серверної інфраструктури.

2.1.8 Site adaption requirements (Вимоги до адаптації на місці)

Для коректного розгортання системи у серверному середовищі необхідно виконати ряд попередніх налаштувань та ініціалізацій даних:

Ініціалізація та системні налаштування:

- Схема бази даних: перед першим запуском системи на сервері бази даних мають бути створені відповідні таблиці та індекси. Також необхідно виконати скрипти міграцій для ініціалізації сутностей системи.

- Конфігураційні змінні: для роботи системи в середовищі сервера повинні бути налаштовані змінні оточення, що включають в себе рядки підключення до БД та секретні ключі для підпису токенів авторизації.

Особливості інстляції та модифікації:

- Налаштування API-ключів: оскільки система використовує зовнішні джерела метаданих, при інсталяції необхідно зареєструвати та впровадити ключі доступу до сторонніх API.

- Локалізація та часові пояси: серверне середовище повинно бути сконфігуроване на використання UTC для коректної синхронзації активності користувачів з різних географічних регіонів.

2.2 Product functions (Функції продукту)

Функціонал системи охоплює повний цикл взаємодії користувача з медіаконтентом - від пошуку до отримання соціального визнання за якісні відгуки. Основні функції системи:

1. Керування персональною бібліотекою контенту:

- Пошук та перегляд контенту через інтегрований каталог;
- Створення персоналізованих списків, наприклад “Переглянуто”, “Планую ознайомитись”;
- Персоналізація профілю, що включає перегляду власних рецензій;

2. Рецензування контенту:

- Оцінювання контенту за 10-бальною шкалою;
- Написання рецензій з розгорнутою думкою щодо того чи іншого контенту;
- Оцінка інших рецензій (лайк/дизлайк);

3. Інтелектуальна навігація та стрічка активності:

- Персоналізована стрічка: відображення нових оцінок та рецензій від користувачів зі схожими смаками;
- Система рекомендацій: пропонування нового контенту на основі попередніх уподобань користувача.

4. Система мотивації та гейміфікація:

- Прогрес користувача: а саме нарахування балів за написання відгуків;
- Досягнення: отримання віртуальних нагород за різного виду активність;
- Рейтинг експертності: формування рівня довіри до рецензій на основі реакції інших користувачів.

2.3 User Characteristics (Характеристики користувачів)

Цільовою аудиторією платформи RateIt є досить широка ланка користувачів, віком від 18 до 60 років, будь-якої статі, професії та соціального положення. Типовий користувач має базові навички роботи з веббраузерами та мобільними застосунками, проте не обов'язково володіє глибокими знаннями в галузі ІТ. Оскільки система орієнтована на написання рецензій, важливими характеристиками користувачів є загальна грамотність та можливість формулювати змістовні відгуки. Такий різноплановий склад аудиторії вимагає від системи максимальної простоти у використанні для виконання базових дій.

2.4 Constraints (Обмеження)

Система “RateIt” обмежена насамперед правовими та технічними стандартами, що визначають вибір архітектури. Система має відповідати політиці конфіденційності та вимогам GDPR, що зобов'язує розробників використовувати шифрування HTTPS та безпечне хешування паролів. Крім того, розробка критично залежить від стабільності та лімітів сторонніх API, що змушує впроваджувати механізми кешування та обробки помилок на випадок збоїв у зовнішніх сервісах.

Експлуатаційні обмеження стосуються надійності та паралельної обробки запитів від великої кількості користувачів. Розробник має гарантувати цілісність транзакцій у базі даних, щоб уникнути конфліктів при одночасному оцінюванні об'єктів або публікацій рецензій. Ці фактори формують технічні межі, у яких реалізуються всі подальші функціональні вимоги продукту.

2.5 Assumptions and Dependencies (Припущення та залежності)

Успішна реалізація та функціонування платформи “RateIt” ґрунтується на низці припущень щодо зовнішнього середовища та залежностей від сторонніх технологій. Основним припущенням є безперебійний доступ до зовнішніх баз метаданих (TMDb, IGDB тощо) та збереження їхнього поточного формату передачі даних. Будь-які радикальні зміни в їхніх API або перехід на виключно платну основу вимагатимуть перегляду функціональних вимог до наповнення каталогу. Також передбачається, що цільова аудиторія має стабільне інтернет-з'єднання, а сучасні браузері продовжуватимуть підтримувати стандарти HTML5 та ECMAScript, необхідні для коректної роботи фронтенд-частини застосунку.

Крім того, проект має критичну залежність від доступності хмарної інфраструктури для розміщення бази даних та серверної логіки. Ми припускаємо, що обраний нами хостинг забезпечуватиме необхідну пропускну здатність та інструменти для резервного копіювання. У разі відсутності підтримки необхідної версії середовища виконання на боці сервера, архітектура системи та терміни розгортання підлягатимуть корекції. Таким чином, стабільність зазначених факторів є базовою умовою для коректної роботи застосунку.

2.6 Apportioning of Requirements (Розподіл вимог)

Через обмежені часові рамки роботи та необхідність першочергового базового запуску функціонала, вимоги системи розділені на два етапи. У першій версії MVP зусилля будуть зосереджені на головній функціональності системи.

Функції, реалізація яких планується в першій версії:

- Перегляд та пошук контенту доступного на сайті;
- Створення профілю користувача та аутентифікація;
- Можливість виставляти оцінки та писати текстові рецензії;
- Формування персональних списків (“Перуглянуто”, “В планах”).

Функції, які можуть бути відкладені до наступних ітерацій:

- Розширена система гейміфікації (складні досягнення та рівні експертності);
- Соціальні функції, наприклад, створення групових обговорень для користувачів;
- Мобільний застосунок (на першому етапі фокус залишається на браузерній версії).

3 SPECIFIC REQUIREMENTS (СПЕЦИФІЧНІ ВИМОГИ)

3.1 External Interfaces (Зовнішні інтерфейси)

Цей розділ містить детальний опис усіх вхідних і вихідних даних програмної системи. Він доповнює опис інтерфейсів із розділу 2.

3.1.1 User Interface (Інтерфейс користувача)

Система повинна надавати адаптивний веб-інтерфейс із графічним відображенням (GUI), доступний через сучасні браузері (Chrome, Firefox, Safari, Edge). Усі основні дії (пошук, оцінювання, написання рецензії) повинні бути доступні не більше ніж у два кліки з будь-якої сторінки. Інтерфейс повинен підтримувати роздільну здатність від 320 пікселів (мобільні пристрої) до 2560 пікселів (десктоп) та відповідати стандарту доступності WCAG 2.1 рівня AA.

3.1.2 API-інтерфейс

Система повинна надавати API через захищений протокол HTTPS. Усі запити та відповіді повинні використовувати формат JSON із кодуванням UTF-8. API повинен повертати стандартні HTTP-коди статусу (200, 201, 400, 401, 403, 404, 500). Автентифікація здійснюється за допомогою JWT токенів, що передаються в заголовку Authorization.

3.1.3 Інтерфейс зовнішніх джерел даних

Система повинна отримувати дані з API зовнішніх сервісів: TMDb API (v3) - для метаданих фільмів і серіалів, IGDB API (v4) - для даних про відеоігри, Google Books API (v1) - для інформації про книги. Усі відповіді зовнішніх API повинні валідуватись і кешуватись для зменшення залежності від зовнішніх сервісів. Вхідні дані: HTTP GET-запит із ключем API та параметрами запиту. Вихідні дані: нормалізований JSON-об'єкт відповідно до внутрішньої схеми MediaItem.

3.2 Functions (Функції)

Функціональні вимоги визначають основні дії, які система повинна виконувати при обробці вхідних даних і генерації вихідних результатів. Кожна вимога ідентифікована унікальним кодом.

ФВ-01: Реєстрація та автентифікація користувачів

Система повинна дозволяти користувачам реєструватися за допомогою електронної пошти та пароля або через OAuth 2.0 (Google, Facebook). Система повинна перевіряти формат електронної пошти та вимагати складності пароля (мінімум 8 символів). Після успішного входу система повинна видавати JWT-токен доступу (термін дії: 1 година) та токен оновлення (термін дії: 30 діб). Після 5 невдалих спроб входу поспіль система повинна блокувати обліковий запис на 15 хвилин.

ФВ-02: Пошук і перегляд контенту

Система повинна забезпечувати єдиний пошук по всіх підтримуваних типах медіаконтенту (фільми, ігри, книги, музика). Результати пошуку з локального кешу повинні повертатися протягом 2 секунд. Система повинна підтримувати фільтрацію за типом медіа, жанром, роком випуску та середнім рейтингом. Пошуковий запит повинен містити від 2 до 200 символів.

ФВ-03: Виставлення рейтингу та написання рецензій

Автентифікований користувач повинен мати можливість виставити числовий рейтинг від 1 до 10 (ціле число) будь-якому медіаоб'єкту. Користувач може виставити лише один рейтинг на медіаоб'єкт; повторне виставлення перезаписує попереднє значення. Автентифікований користувач може за бажанням додати текстову рецензію (від 10 до 5000 символів) до свого рейтингу. Система повинна перераховувати та оновлювати середній рейтинг медіаоб'єкта протягом 5 секунд після кожного нового виставлення.

ФВ-04: Управління персональною бібліотекою

Система повинна дозволяти автентифікованим користувачам додавати медіаоб'єкти до іменованих персональних списків. Стандартні списки: "Переглянуто/Зіграно/Прочитано" та "Планую переглянути/зіграти/прочитати". Користувачі повинні мати можливість створювати власні іменовані списки. Кожен список повинен відображати назву, тип медіа, обкладинку та рейтинг користувача для кожного елемента.

ФВ-05: Система гейміфікації

Система повинна нараховувати бали за активність: 5 балів за виставлений рейтинг, 20 балів за написану рецензію, 2 бали за отриману вподобайку на рецензію. Система повинна автоматично видавати значки досягнень при досягненні визначених порогів (наприклад, «Перша рецензія» - після написання першої рецензії). Загальна кількість балів та зароблені значки повинні відображатися на публічній сторінці профілю.

ФВ-06: Стрічка активності

Система повинна відображати хронологічну стрічку активності на головній сторінці користувача із новими рейтингами та рецензіями від користувачів із подібними смаками. Стрічка повинна реалізовувати пагінацію блоками по 20 елементів. Система повинна підтримувати реакції "лайк" і "дизлайк" на окремі рецензії у стрічці.

3.3 Performance Requirements (Вимоги до продуктивності)

ПВ-01: Система повинна підтримувати щонайменше 500 одночасних користувачів без погіршення часу відгуку.

ПВ-02: 95% усіх запитів на завантаження сторінок повинні виконуватись менш ніж за 2 секунди при нормальному навантаженні (до 500 одночасних користувачів).

ПВ-03: Відповіді API на кешовані пошукові запити повинні надходити менш ніж за 500 мс у 99% випадків.

ПВ-04: Система повинна обробляти операції запису до бази даних (виставлення рейтингів, публікації рецензій) зі швидкістю не менше 100 транзакцій на секунду.

ПВ-05: Загальний розмір зібраного пакету фронтенд-застосунку не повинен перевищувати 2 МБ (стиснений) для забезпечення швидкого початкового завантаження на мобільних пристроях.

3.4 Logical Database Requirements (Логічні вимоги до бази даних)

База даних повинна зберігати такі основні сутності та їхні зв'язки:

User (Користувач): email, password, firstName, lastName, age. Поле email є унікальним ідентифікатором користувача в системі. Пароль зберігається у хешованому вигляді.

Content (Медіаконтент): Система підтримує шість типів контенту, кожен із власним набором полів:

Film: Title, Description, ReleaseDate, ReleaseYear, ImageUrl, FilmDirector, FilmWriters, FilmMainCast, FilmBudget (int), FilmBoxOffice (int), FilmRuntimeInMinutes (int), FilmCountryOfOrigin, FilmAwards, FilmGenres (array).

TVShow: Title, Description, ReleaseDate, ReleaseYear, ImageUrl, TVShowCreators, TVShowDirector, TVShowMainCast, TVShowTotalSeasons (int), TVShowTotalEpisodes (int), TVShowNetworks, TVShowEndDate, TVShowGenres (array).

Episode: Title, Description, ReleaseDate, ReleaseYear, ImageUrl, EpisodeTVShowId (FK → TVShow), EpisodeSeasonNumber (int), EpisodeNumber (int), EpisodesTotalNumber (int), EpisodeRuntimeInMinutes (int).

Music: Title, Description, ReleaseDate, ReleaseYear, ImageUrl, MusicArtist, MusicAlbum, MusicDurationInSeconds (int), MusicLabel, MusicLanguage, MusicGenres (array).

Game: Title, Description, ReleaseDate, ReleaseYear, ImageUrl, GameDeveloper, GamePublisher, GamePlatforms (array), GameGenres (array).

Book: Title, Description, ReleaseDate, ReleaseYear, ImageUrl, BookAuthor, BookPublisher, BookOriginalLanguage, BookPages (int), BookGenres (array).

Усі типи контенту мають спільні базові поля: Title, Description, ReleaseDate, ReleaseYear, ImageUrl. Специфічні поля зберігаються окремо для кожного типу.

Rating (Рейтинг): Прив'язаний до пари (User, Content). Зберігає числову оцінку користувача (score, INT 1–10), а також дати створення та оновлення. На пару (user_id, content_id) накладається унікальне обмеження — один користувач може мати лише одну оцінку для одного об'єкта.

Review (Рецензія): Прив'язана до Rating. Містить текстове поле body (TEXT, макс. 5000 символів), лічильники likes_count та dislikes_count, дати створення та оновлення.

UserList (Список користувача): Дозволяє користувачам групувати контент у власні списки. Містить назву списку (VARCHAR 100) та прапорець is_default для системних списків (наприклад, «Переглянуто», «Хочу переглянути»).

Achievement (Досягнення): Зберігає інформацію про досягнення системи гейміфікації: унікальний код, назву, опис, іконку та поріг балів для отримання.

Обмеження цілісності: усі зовнішні ключі для даних, що належать користувачу, використовують CASCADE DELETE. Поле середньої оцінки контенту оновлюється автоматично через тригер бази даних при INSERT/UPDATE/DELETE записів Rating. Episode обов'язково має посилатися на існуючий TVShow через поле EpisodeTVShowId.

3.5 Design Constraints (Конструктивні обмеження)

КО-01: Серверна частина повинна працювати в межах бюджету оперативної пам'яті 512 МБ RAM для середовища розгортання MVP.

КО-02: Усі завантажувані користувачем медіафайли (аватари, зображення до рецензій) не повинні перевищувати 2 МБ на файл. Дозволені формати: лише JPEG та PNG.

КО-03: Архітектура системи повинна відповідати принципу розділення клієнта і сервера. Фронтенд і бекенд повинні взаємодіяти виключно через API, без прямого доступу клієнта до бази даних.

3.5.1 Standards Compliance (Відповідність стандартам)

ВС-01: Система повинна відповідати вимогам Загального регламенту про захист даних (GDPR, Регламент ЄС 2016/679). Це передбачає явну згоду користувача на збір даних, право на видалення та право на експорт даних.

ВС-02: Інтерфейс користувача повинен відповідати вимогам доступності WCAG 2.1 рівня AA (рекомендація W3C, червень 2018).

ВС-03: Усі HTTPS-з'єднання повинні використовувати TLS 1.2 або вище (RFC 5246 / RFC 8446). Протоколи SSLv3 і TLS 1.0 повинні бути явно вимкнені.

ВС-04: Паролі користувачів повинні зберігатися у вигляді хешу bcrypt із мінімальним коефіцієнтом складності 12 (відповідно до настанови NIST SP 800-63B).

ВС-05: Цей SRS-документ підготовлено відповідно до IEEE Std 830-1998.

3.6 Software System Attributes (Атрибути програмної системи)

3.6.1 Reliability (Надійність)

НД-01: Система повинна досягати середнього напрацювання до відмови (MTBF) не менше 720 годин (30 діб) для основного функціоналу (автентифікація, оцінювання, рецензії).

НД-02: У разі відмови зовнішнього API (TMDb, IGDB, Google Books) система повинна обслуговувати кешовані дані та не переходити у стан збою. Якщо після 3 повторних спроб свіжі дані не вдається отримати, користувачеві повинне відображатися зрозуміле повідомлення про помилку.

НД-03: Усі операції запису до бази даних повинні виконуватися в межах транзакцій для забезпечення цілісності даних. Часткові записи, що виникли внаслідок збою системи, повинні повністю відкатуватись.

3.6.2 Availability (Доступність)

ДС-01: Система повинна забезпечувати цільовий показник доступності 99,5% на місяць, що допускає не більше приблизно 3,6 години незапланованих простоїв на місяць.

ДС-02: Планові технічні роботи повинні проводитися у години мінімальної активності (02:00-04:00 UTC) і не перевищувати 2 години. Користувачі повинні отримувати сповіщення через банер у застосунку не менше ніж за 24 години.

ДС-03: Система повинна відновлюватись після повної відмови сервера. Цільовий час відновлення (RTO) - не більше 4 годин; цільова точка відновлення (RPO) - не більше 24 годин, що забезпечується щоденним автоматичним резервним копіюванням бази даних.

3.6.3 Security (Безпека)

БЗ-01: Уся передача даних між клієнтом і сервером повинна шифруватися за допомогою HTTPS/TLS 1.2+. Незашифровані HTTP-з'єднання повинні автоматично перенаправлятися на HTTPS.

БЗ-02: Система повинна захищатися від загроз OWASP Top 10, зокрема: SQL-ін'єкцій (параметризовані запити), міжсайтового скриптингу/XSS (кодування виведення) та підробки міжсайтових запитів/CSRF (перевірка токена).

БЗ-03: API повинен реалізовувати обмеження частоти запитів (rate limiting): не більше 100 запитів на хвилину для автентифікованого користувача та 20 запитів на хвилину для неавтентифікованої IP-адреси.

БЗ-04: Журнал подій безпеки повинен фіксувати події автентифікації (успішні входи, невдалі спроби, виходи) із зазначенням часу, ідентифікатора користувача та IP-адреси. Записи журналу повинні зберігатися не менше 90 діб.

3.6.4 Maintainability (Супроводжуваність)

СП-01: Кодова база повинна бути організована у відокремлені модулі відповідно до функціональних доменів (auth, users, media, ratings, reviews, gamification). Міжмодульні залежності повинні бути мінімізовані та задокументовані.

СП-02: Усі публічні кінцеві точки API та сервісні інтерфейси повинні мати вбудовану документацію. Зміни схеми бази даних повинні управлятися виключно через версійовані скрипти міграцій.

СП-03: Рівень інтеграції зовнішніх API метаданих (TMDB, IGDB, Google Books) повинен бути ізольований за допомогою шаблону адаптера, щоб заміна або додавання зовнішнього джерела не вимагала змін у основній бізнес-логіці.

3.6.5 Portability (Переносимість)

ПЕ-01: Увесь бекенд-застосунок повинен бути контейнеризований за допомогою Docker. Конфігурація docker-compose повинна забезпечувати локальне розгортання однією командою на будь-якій ОС, що підтримує Docker Engine.

ПЕ-02: Фронтенд-застосунок повинен коректно функціонувати у двох останніх стабільних версіях Chrome, Firefox, Safari та Microsoft Edge.

ПЕ-03: Усі параметри, що залежать від середовища (рядки підключення до БД, ключі API, секрети JWT), повинні керуватися виключно через змінні оточення та не повинні жорстко кодуватися в початковому коді.

3.7 Organizing the Specific Requirements (Організація специфічних вимог)

Специфічні вимоги платформи «RateIt» у розділі 3.2 організовано за принципом ознак (Feature), оскільки система має шість чітко визначених, зовнішньо видимих функціональних ознак, які незалежно тестуються та поставляються. Нижче наведено опис усіх альтернативних підходів до організації та їхнє застосування до даної системи.

3.7.1 System Mode (Режим системи)

Система «RateIt» функціонує у двох режимах. У користувацькому режимі (User Mode) звичайні автентифіковані користувачі можуть переглядати контент, виставляти рейтинги та рецензії, керувати персональною бібліотекою та взаємодіяти зі стрічкою активності. В адміністративному режимі (Administrator Mode) привілейовані користувачі мають доступ до модерації контенту, управління скаргами, редагування метаданих медіаоб'єктів та перегляду системних звітів. Вимоги для кожного режиму виокремлені у відповідних підрозділах розділу 3.2.

3.7.2 User Class (Клас користувача)

Система розрізняє три класи користувачів:

Guest (Гість) - неавтентифікований відвідувач; може переглядати контент, бачити середні рейтинги та читати публічні рецензії, не може виставляти рейтинги та писати рецензії.

Registered User (Зареєстрований користувач) - власник автентифікованого облікового запису; має доступ до всього функціоналу, описаного у ФВ-01 - ФВ-06.

Administrator (Адміністратор) - співробітник платформи, який має всі можливості зареєстрованого користувача та інструменти модерації контенту, управління користувачами та системні звіти.

3.7.3 Objects (Об'єкти)

Основні об'єкти, що мають відповідники в системі:

User - зареєстрований користувач із профілем (email, firstName, lastName, age). Є власником об'єктів Rating, Review та UserList. Накопичує досягнення через рушій гейміфікації.

Film, TVShow, Episode, Music, Game, Book - шість типів контенту, що каталогізуються в системі. Кожен тип має спільні базові поля (Title, Description, ReleaseDate, ReleaseYear, ImageUrl) та власний набір специфічних атрибутів. Підтримувані операції: пошук, відображення, обчислення агрегованого рейтингу.

Rating - оцінка, що належить одному User та одному об'єкту контенту. Один користувач може виставити лише одну оцінку для одного об'єкта. Операції: створення, оновлення.

Review - текстова рецензія, прив'язана до одного Rating. Операції: створення, оновлення, видалення, отримання реакцій (лайки/дизлайки).

UserList - користувацький список для групування контенту (наприклад, «Переглянуто», «Хочу переглянути»). Належить одному User.

Achievement - досягнення, визначене системою. Видається User автоматично при досягненні порогу активності. Містить назву, опис, іконку та порогове значення балів.

3.7.4 Feature (Ознака)

Вимоги у розділі 3.2 організовано за ознаками, оскільки «RateIt» має шість добре визначених, зовнішньо видимих наборів функцій, що незалежно тестуються та поставляються. Така організація забезпечує максимальну відстежуваність між бізнес-вимогами та умовами тестування і узгоджується з ітеративним планом розробки, описаним у розділі 2.6. Шість ознак: (1) Реєстрація та автентифікація; (2) Пошук і перегляд контенту; (3) Виставлення рейтингу та рецензії; (4) Управління персональною бібліотекою; (5) Система гейміфікації; (6) Стрічка активності.

3.7.5 Stimulus (Стимул)

Ключові стимули, що керують поведінкою системи: надсилання пошукового запиту користувачем (ініціює отримання контенту з кешу або зовнішнього API); виставлення рейтингу або написання рецензії (ініціює запис до БД, перерахунок середнього та нарахування балів гейміфікації); клік лайку/дизлайку на рецензію (ініціює оновлення лічильника реакцій і оновлення стрічки); тригер запланованого фонових завдання (ініціює оновлення застарілих записів MediaItem у кеші). Організація за стимулом є доречною для суто подійно-керованих систем; для «RateIt» перевагу надано організації за ознаками, оскільки вона краще слугує команді розробників.

3.7.6 Response (Відгук)

Критичні відгуки системи та пов'язані вимоги: відгук на успішну автентифікацію - система повертає JWT-токен і перенаправляє на домашню стрічку (ФВ-01); відгук із результатами пошуку - система повертає посторінковий JSON-масив підсумків MediaItem протягом 2 секунд (ФВ-02, ПВ-02); підтвердження виставлення рейтингу - система підтверджує збереження, оновлює відображуваний середній рейтинг і запускає логіку гейміфікації (ФВ-03, ФВ-05); відгук на помилку - всі

стани помилок повертають структурований JSON-об'єкт із машинозчитуваним кодом помилки та зрозумілим повідомленням (ФВ-01 - ФВ-06).

3.7.7 Functional Hierarchy (Функціональна ієрархія)

На верхньому рівні всі функції системи поділяються на два домени: Взаємодія з контентом (пошук, перегляд, оцінювання, рецензування, управління списками) та Платформні сервіси (автентифікація, гейміфікація, стрічка активності). У межах «Взаємодії з контентом» основний потік даних: зовнішній API → рівень кешу → функції пошуку/перегляду → сторінка деталей медіаоб'єкта → виставлення рейтингу/рецензії. У межах «Платформних сервісів»: подія дії користувача → рушій гейміфікації → оновлення балів/досягнень → відображення у профілі та стрічці. Ця ієрархія визначає декомпозицію бекенд-модулів і діаграми потоків даних у проектній документації.

4 ПРОЦЕС УПРАВЛІННЯ ЗМІНАМИ (CHANGE MANAGEMENT PROCESS)

4.1 Загальні положення

Будь-яка зміна до цього документа проходить через визначену процедуру. Це дозволяє уникнути плутанини та зберегти актуальність вимог на всіх етапах роботи над проектом.

4.2 Ролі та відповідальності

У процесі управління змінами задіяні такі ролі:

Ініціатор зміни - будь-який член команди або замовник, який вважає, що певну вимогу потрібно додати або змінити.

Відповідальний за вимоги - веде журнал змін та оновлює SRS після прийнятих рішень.

Усі рішення щодо змін приймаються спільно.

4.3 Подання запиту на зміну

Якщо хтось хоче змінити вимогу - він має подати запит у письмовій формі. Телефонний дзвінок або усне прохання не вважається офіційним запитом і не тягне за собою жодних змін до документа.

Прийнятні способи подання:

1. Електронна пошта на адресу відповідального за вимоги;
2. Issue у GitHub репозиторії проекту.

Запит має містити: що саме пропонується змінити, чому це потрібно, яких вимог стосується та як це може вплинути на строки.

4.4 Оцінка та прийняття рішення

Після отримання запиту відповідальний за вимоги реєструє його в журналі змін із присвоєнням номера (CR-001, CR-002 і т.д.) протягом одного робочого дня.

Обидва члени команди разом розглядають запит - як правило, на черговій спільній зустрічі. Оцінюється, наскільки зміна реалістична, як вона впливає на вже виконану роботу та чи відповідає цілям MVP. Рішення приймається лише за згоди обох - жоден із членів команди, і тим більше замовник, не може самостійно затвердити зміну.

4.5 Впровадження змін

Якщо зміну схвалено - відповідальний за вимоги оновлює SRS: редагує потрібні розділи, змінює номер версії (незначна зміна - +0.1, суттєва - +1.0) та фіксує дату. Оновлений документ надсилається замовнику та зберігається в репозиторії.

Якщо запит відхилено - ініціатор отримує коротке пояснення. Усі запити зберігаються в журналі змін до кінця проєкту.

5 DOCUMENT APPROVALS (СХВАЛЕННЯ ДОКУМЕНТА)

Нижченаведені особи ознайомилися з цим документом SRS та підтвердили його відповідність цілям і вимогам проєкту «RateIt». Підписавши цей документ, кожна особа підтверджує, що зміст специфікації є точним, повним і достатнім для початку розробки.

Таблиця 4 – Document approvals (схвалення документа)

ПІБ	Роль	Підпис	Дата
Кулаков Сергій Михайлович	Back End Розробник	_____	10.04.2026
Єрмоленко Олександр Станіславович	Front End Розробник	_____	10.04.2026

6 ДОПОМІЖНА ІНФОРМАЦІЯ (SUPPORTING INFORMATION)

Додаток А. Журнал змін документа (Change Log)

Версія	Дата	Автор	Опис змін
1.0	10.04.2026	Кулаков С. М., Єрмоленко О. С.	Початкова версія документа. Розділи 1–6 повністю заповнені.